

Wholesale - Clustering, clasificación y análisis de sensibilidad

Resumen

En este trabajo se analiza el dataset `wholesale`, del paquete `{tclust}`. La base contiene 440 clientes de un distribuidor mayorista. Las variables numéricas representan el gasto anual en seis categorías de productos: `Fresh`, `Milk`, `Grocery`, `Frozen`, `Detergent` y `Delicatessen`. Además, hay dos variables categóricas: `Region` y `Channel`.

El recorrido del análisis es el siguiente:

1. Primero se exploran los datos para entender su forma, sus valores extremos y la relación con `Region` y `Channel`.
2. Luego se transforman las variables numéricas con `log1p()` y se escalan para poder usar PCA y métodos de clustering.
3. Se aplica PCA para reducir la dimensionalidad y observar si aparecen patrones vinculados con `Region` o `Channel`.
4. Se realiza clustering jerárquico y se compara la partición con `Channel`.
5. Se comparan cuatro métodos de clustering: k-means, GMM, DBSCAN y OPTICS.
6. Se ajustan modelos de clasificación supervisada y semisupervisada para predecir `Channel`.
7. Finalmente, se repite el análisis quitando outliers multivariados para ver si las conclusiones se mantienen.

La idea central no es solo correr métodos, sino ver cuáles ayudan a entender la estructura de los datos y cuáles no.

0. Preparación

Cargamos los paquetes y el dataset.

```
library(tclust)
```

Robust Trimmed Clustering (version 2.2-0)

```
library(mclust)
```

Package 'mclust' version 6.1.2

Type 'citation("mclust")' for citing this R package in publications.

```
library(dbSCAN)
```

Adjuntando el paquete: 'dbSCAN'

The following object is masked from 'package:stats':

```
as.dendrogram
```

```
library(ggplot2)
library(factoextra)
```

Welcome to factoextra!

Want to learn more? See two factoextra-related books at <https://www.datanovia.com/en/product/practical-guide-to-principal-component-methods-in-r/>

```
library(cluster)
library(MASS)
library(RColorBrewer)
library(aricode)
library(corrplot)
```

corrplot 0.95 loaded

```
data(wholesale)
```

1. Exploración inicial

1.1 Valores faltantes y separación de variables

Primero revisamos si hay valores faltantes. Luego separamos las variables numéricas de las categóricas. Esto es útil porque PCA, clustering y clasificación van a trabajar principalmente con las variables numéricas, mientras que **Region** y **Channel** se usan para interpretar y comparar resultados.

```
sum(is.na(wholesale))
```

```
[1] 0
```

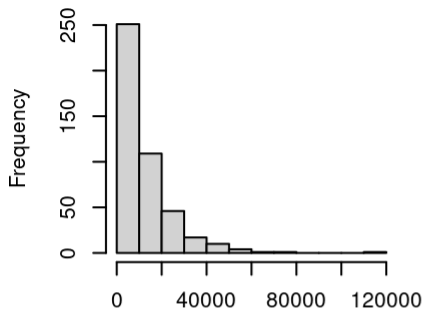
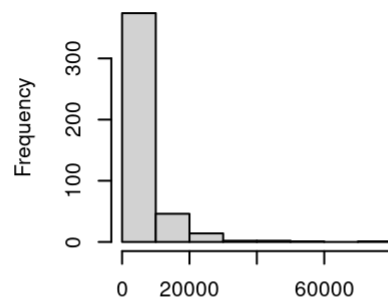
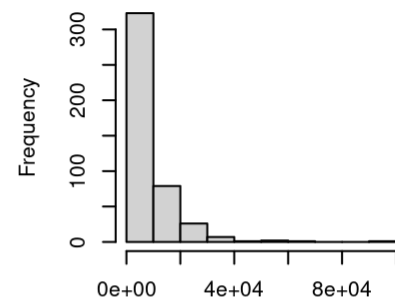
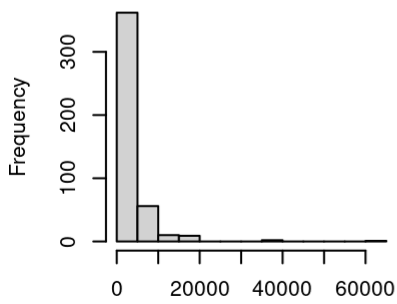
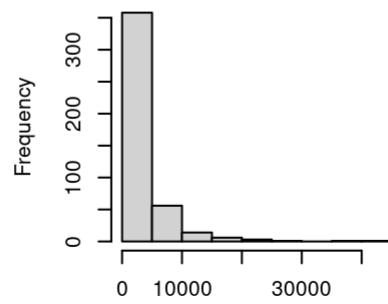
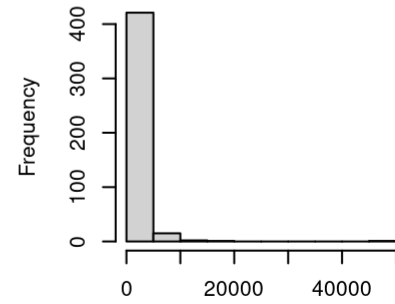
```
wholesale_num <- wholesale[, 2:7]
wholesale_factor <- wholesale[, c(1, 8)]
```

No se observan valores faltantes. Por eso no es necesario imputar ni eliminar observaciones en esta etapa.

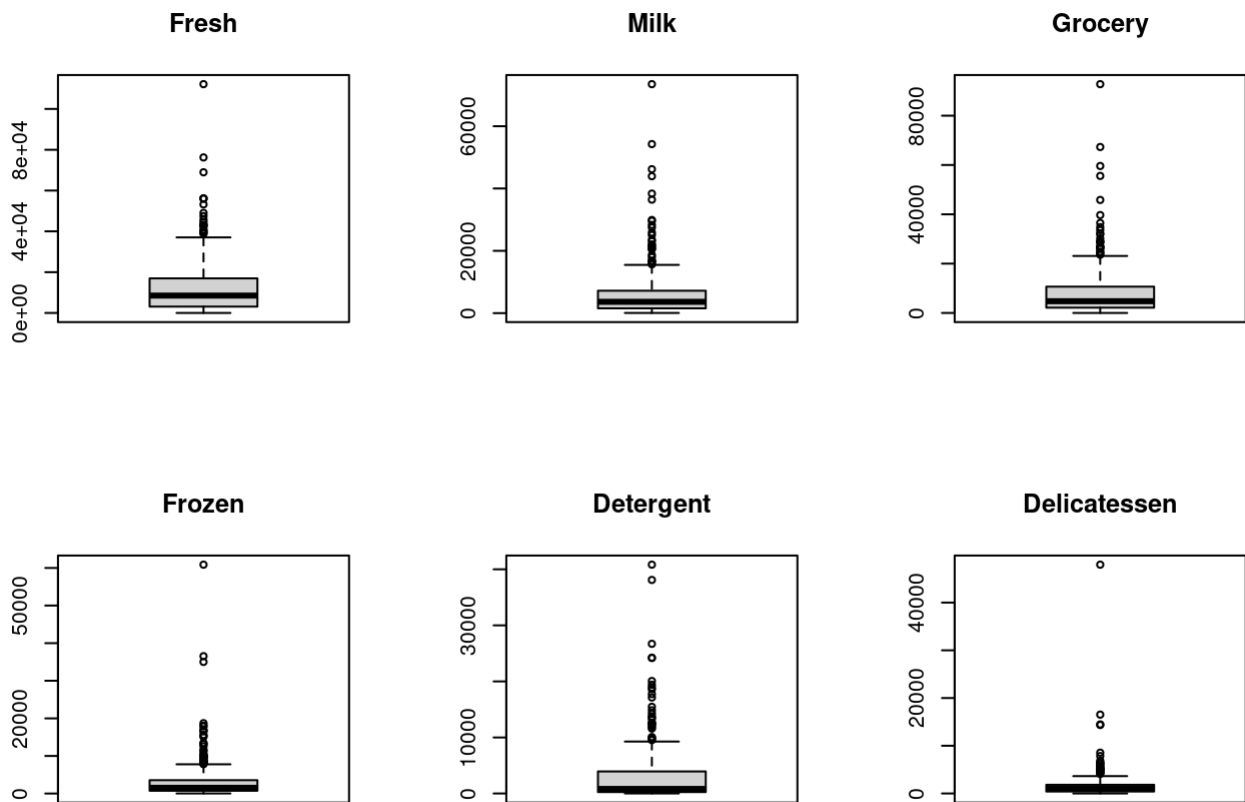
1.2 Distribución de las variables numéricas

Miramos histogramas y boxplots de las variables originales. Esto permite ver si hay asimetría, dispersión y valores extremos.

```
par(mfrow = c(2, 3))
for (i in 1:6) {
  hist(wholesale_num[, i],
       main = colnames(wholesale_num)[i],
       xlab = "")
}
```

Fresh**Milk****Grocery****Frozen****Detergent****Delicatessen**

```
par(mfrow = c(2, 3))
for (i in 1:6) {
  boxplot(wholesale_num[, i],
    main = colnames(wholesale_num)[i])
}
```



```
par(mfrow = c(1, 1))
```

Las variables de gasto presentan distribuciones asimétricas hacia la derecha. La mayoría de los clientes tiene consumos bajos o medios, y unos pocos clientes tienen consumos muy altos.

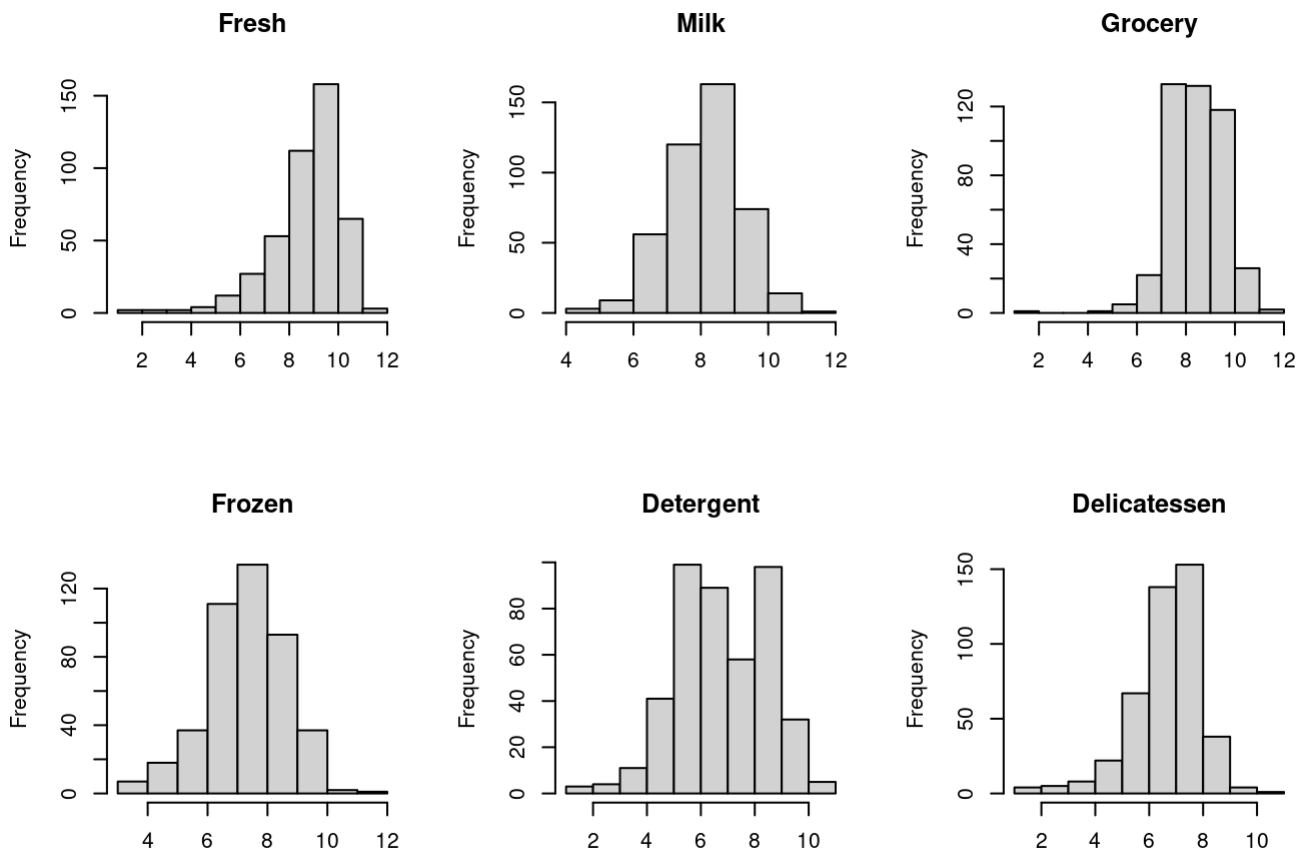
Los boxplots muestran muchos valores extremos. En principio no los tomo automáticamente como errores, porque pueden representar clientes grandes o con un perfil de compra diferente.

1.3 Transformación logarítmica

Como las variables tienen mucha asimetría positiva, aplicamos `log1p()`, que calcula $\log(x + 1)$. Esta transformación reduce el peso de los valores muy altos sin eliminar observaciones.

```
wholesale_log <- log1p(wholesale_num)

par(mfrow = c(2, 3))
for (i in 1:6) {
  hist(wholesale_log[, i],
       main = colnames(wholesale_log)[i],
       xlab = "")
}
```



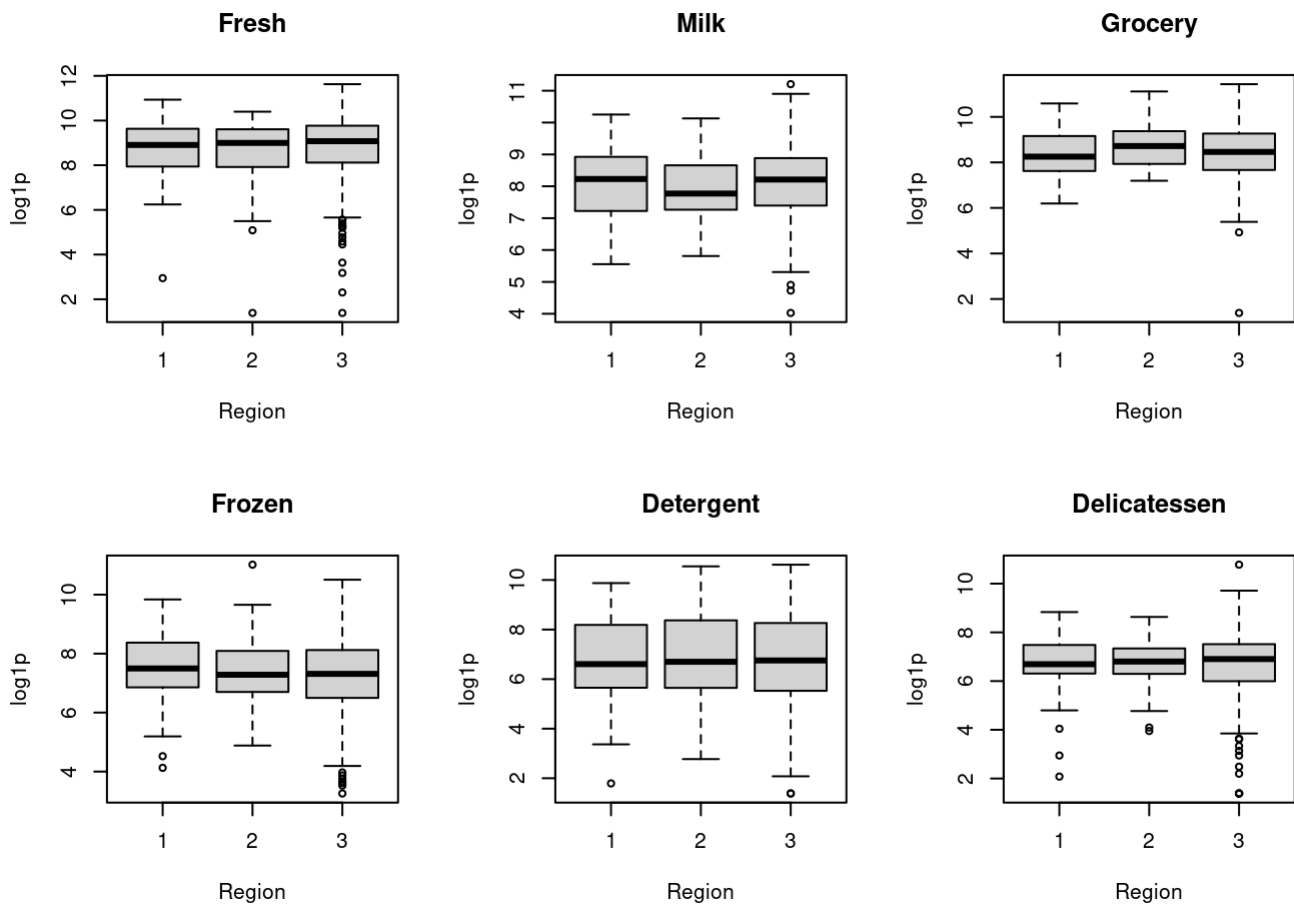
```
par(mfrow = c(1, 1))
```

Después de la transformación logarítmica, las distribuciones quedan más simétricas y son más adecuadas para analizar distancias, PCA y clustering.

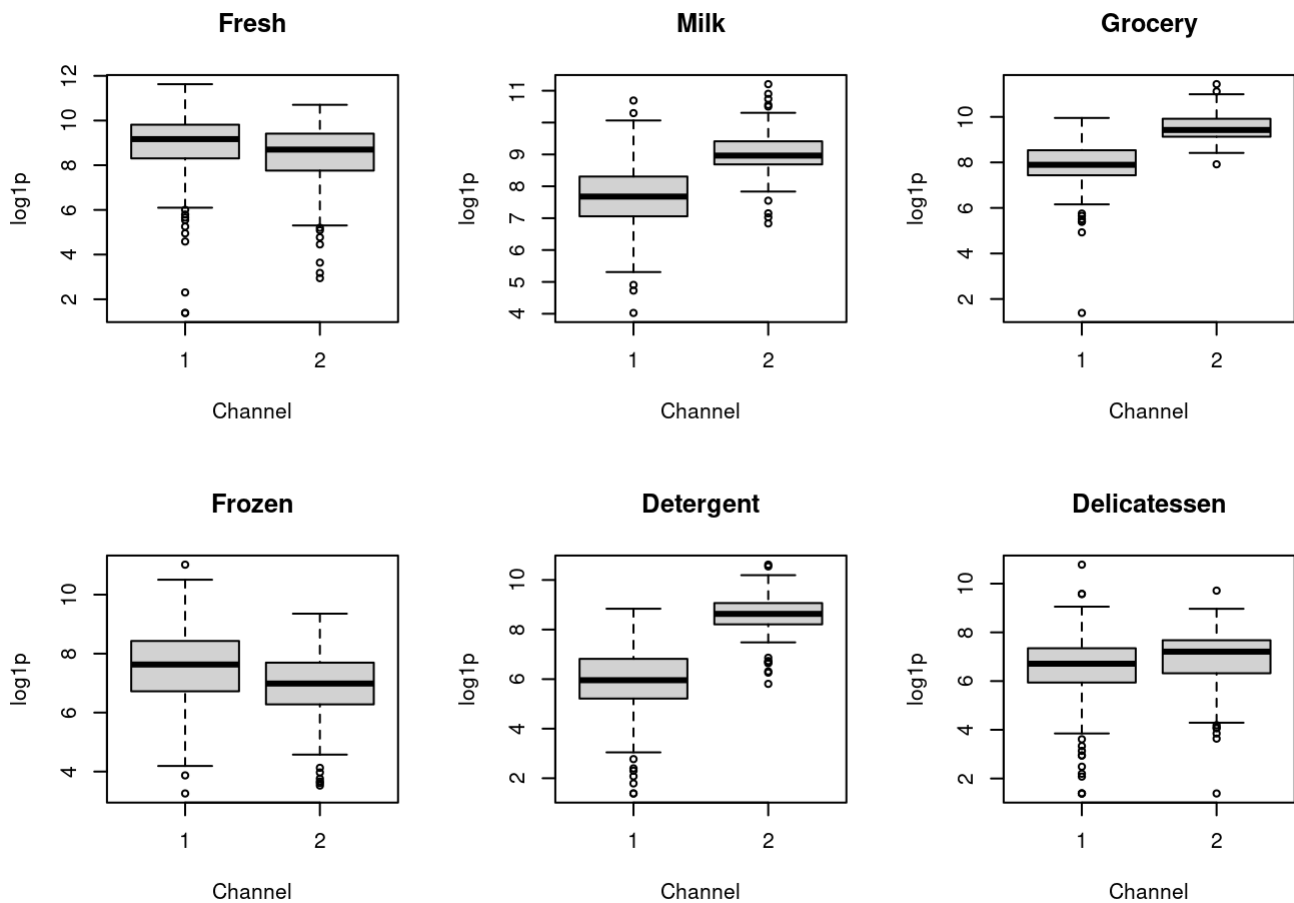
1.4 Comparación por Region y por Channel

Ahora miramos si las variables transformadas cambian según Region y según Channel.

```
par(mfrow = c(2, 3))
for (i in 1:6) {
  boxplot(wholesale_log[, i] ~ wholesale_factor[, "Region"],
    main = colnames(wholesale_num)[i],
    xlab = "Region",
    ylab = "log1p")
}
```



```
par(mfrow = c(2, 3))
for (i in 1:6) {
  boxplot(wholesale_log[, i] ~ wholesale_factor[, "Channel"],
    main = colnames(wholesale_num)[i],
    xlab = "Channel",
    ylab = "log1p")
}
```



```
par(mfrow = c(1, 1))
```

Al comparar por **Region**, las medianas son relativamente similares entre las tres regiones. La región 3 muestra más dispersión, pero también tiene más observaciones.

Al comparar por **Channel**, la diferencia es más clara. El **Channel 2** tiende a tener valores más altos en **Milk**, **Grocery** y **Detergent**. El **Channel 1** muestra más dispersión en algunas variables como **Frozen** y **Delicatessen**.

Esto anticipa que **Channel** podría estar más relacionado que **Region** con la estructura de consumo.

1.5 Balance de las variables categóricas

```
table(wholesale_factor[, "Region"])
```

```
1  2  3
77 47 316
```

```
table(wholesale_factor[, "Channel"])
```

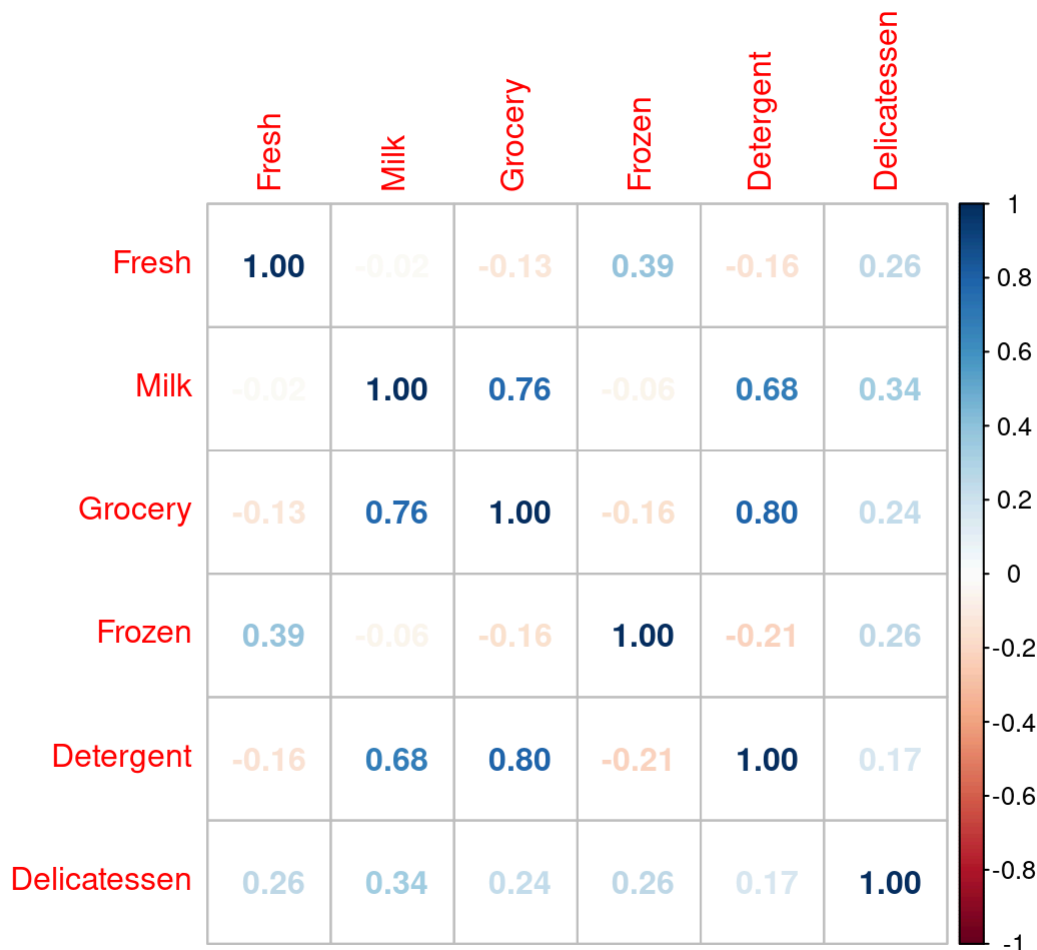
```
1  2
298 142
```

La región 3 y el canal 1 tienen más observaciones, así que los datos no están balanceados. Esto no impide seguir con el análisis, pero después hay que comparar usando proporciones y, en clasificación, mirar también el rendimiento por clase.

1.6 Correlación entre variables

Revisamos la matriz de correlaciones entre variables transformadas.

```
cor_wholesale <- cor(wholesale_log)
corrplot(cor_wholesale, method = "number", type = "full")
```

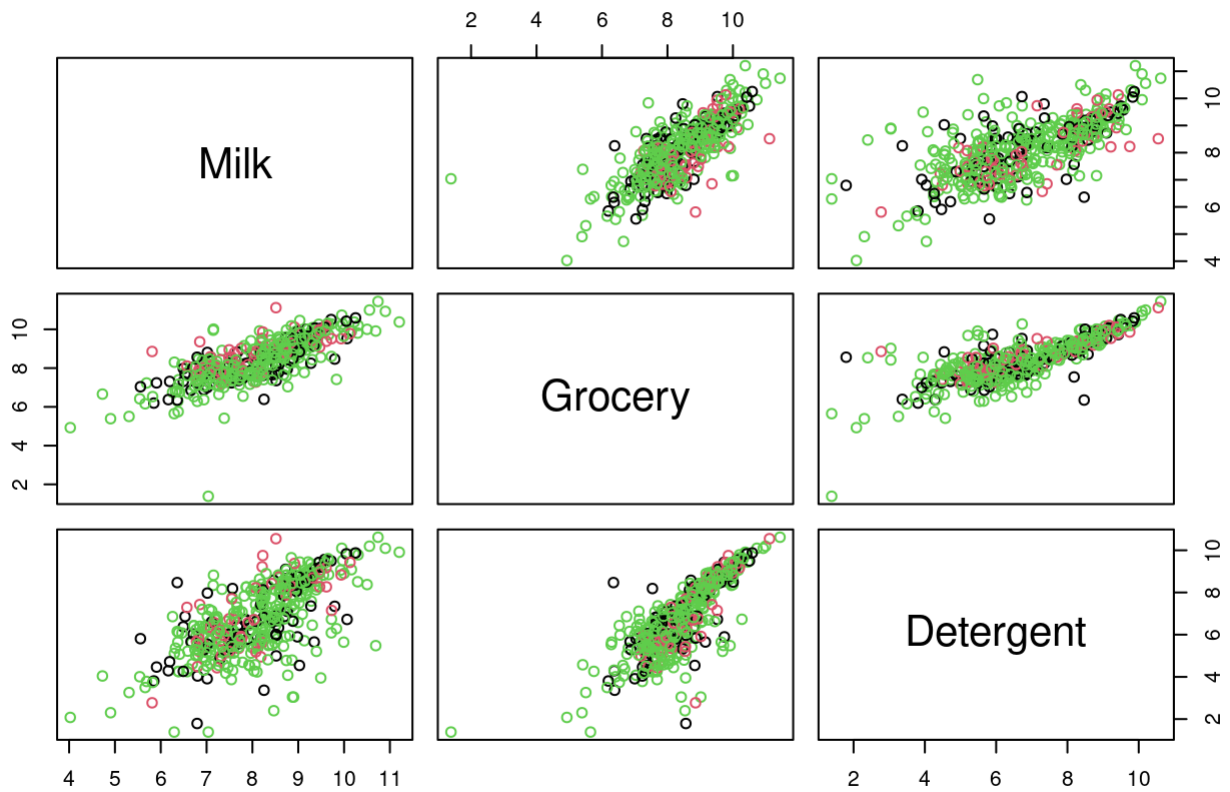


Las correlaciones más fuertes aparecen entre **Grocery**, **Milk** y **Detergent**. Esto indica que estas variables tienden a aumentar juntas: los clientes que compran más **Grocery** suelen comprar más **Milk** y más **Detergent**.

Podemos verlo también con gráficos de pares, coloreando por **Region** y por **Channel**.

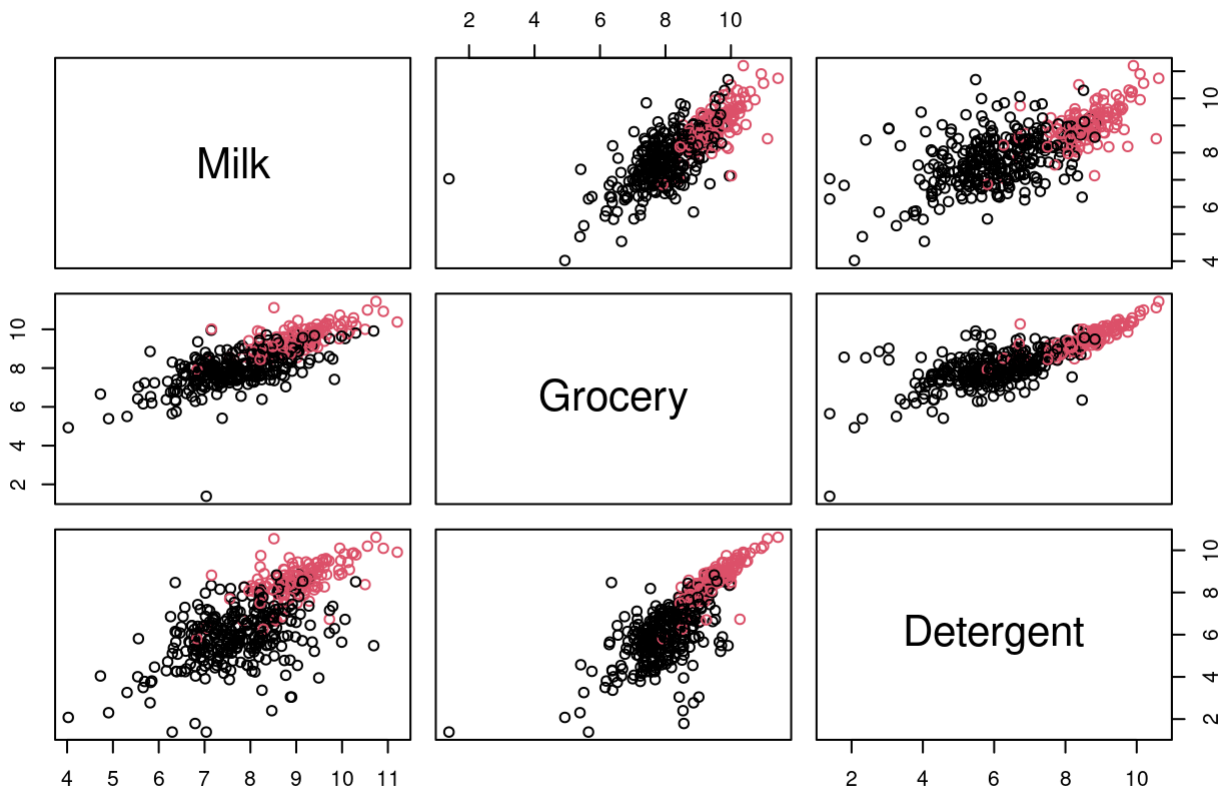
```
pairs(wholesale_log[, c("Milk", "Grocery", "Detergent")],
      col = as.numeric(wholesale_factor[, "Region"]),
      main = "Variables correlacionadas coloreadas por Region")
```


Variables correlacionadas coloreadas por Region



```
pairs(wholesale_log[, c("Milk", "Grocery", "Detergent")],  
      col = as.numeric(wholesale_factor[, "Channel"]),  
      main = "Variables correlacionadas coloreadas por Channel")
```

Variables correlacionadas coloreadas por Channel



Al colorear por **Region**, no aparece una separación clara. Al colorear por **Channel**, sí se ve una diferencia más marcada, sobre todo porque el canal 2 tiende a ubicarse en valores más altos de estas variables.

1.7 Centrado y escalado

Aunque aplicamos logaritmo, las variables todavía tienen medias y desvíos distintos. Por eso se aplica centrado y escalado antes de PCA y clustering.

```
apply(wholesale_log, 2, mean)
```

Fresh	Milk	Grocery	Frozen	Detergent	Delicatessen
8.732813	8.121615	8.442205	7.303128	6.791781	6.671094

```
apply(wholesale_log, 2, sd)
```

Fresh	Milk	Grocery	Frozen	Detergent	Delicatessen
1.470618	1.080635	1.111523	1.281888	1.709519	1.293960

```
wholesale_log_escalado <- scale(wholesale_log,
                                center = TRUE,
                                scale = TRUE)
```

El centrado lleva las variables a media cero y el escalado las lleva a desvío estándar uno. Esto evita que una variable tenga más peso solo por estar en una escala o dispersión mayor.

1.8 Outliers multivariados

Para detectar observaciones alejadas considerando todas las variables al mismo tiempo, usamos distancia de Mahalanobis.

```
centro <- colMeans(wholesale_log_escalado)
S <- cov(wholesale_log_escalado)

dist_maha <- mahalanobis(wholesale_log_escalado,
                        center = centro,
                        cov = S)

limite <- quantile(dist_maha, 0.975)
outliers <- dist_maha > limite
sum(outliers)
```

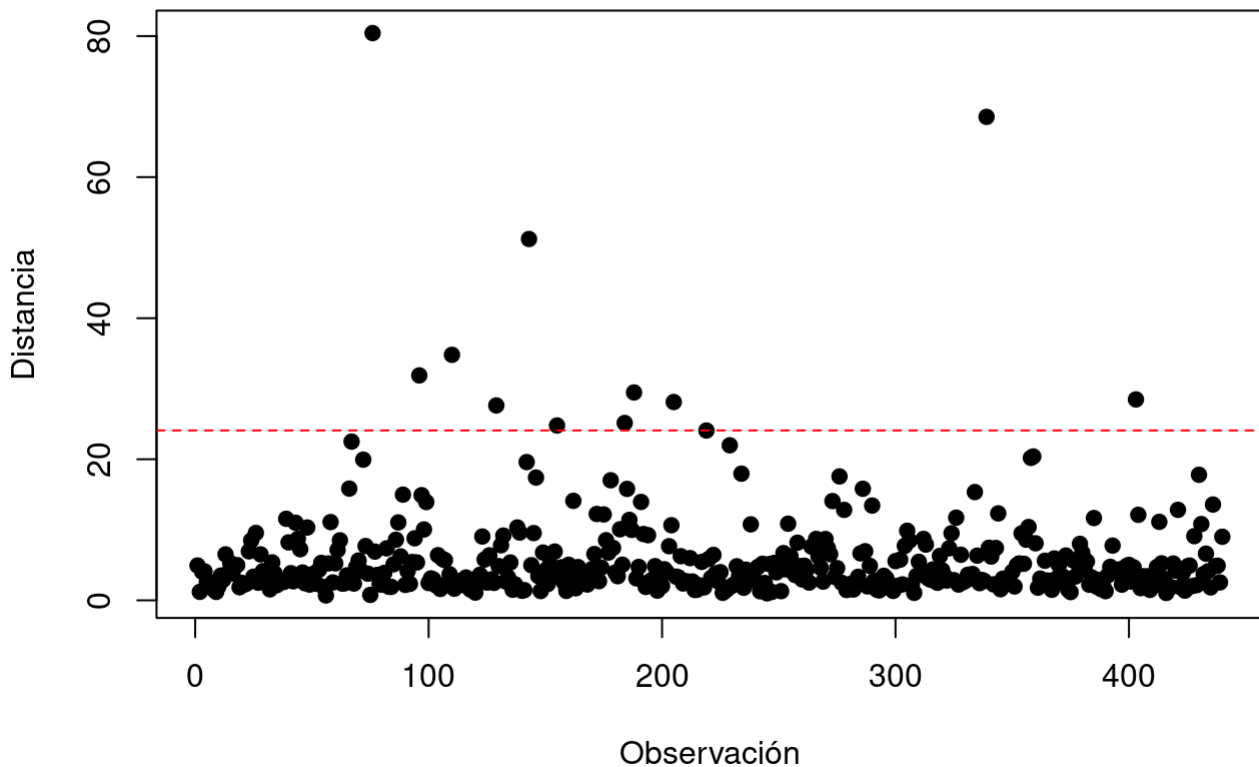
```
[1] 11
```

Se usó como corte el percentil 97.5 de las distancias. Las observaciones marcadas no se eliminan automáticamente, porque pueden representar clientes reales diferentes al resto.

```
wholesale_log_escalado_df <- as.data.frame(wholesale_log_escalado)
wholesale_log_escalado_df$outlier <- outliers
wholesale_log_escalado_df$dist_maha <- dist_maha
wholesale_log_escalado_df$Region <- wholesale_factor$Region
wholesale_log_escalado_df$Channel <- wholesale_factor$Channel

plot(wholesale_log_escalado_df$dist_maha,
     pch = 19,
     main = "Distancia Mahalanobis",
     xlab = "Observación",
     ylab = "Distancia")
abline(h = limite, col = "red", lty = 2)
```

Distancia Mahalanobis



2. Reducción de dimensionalidad

2.1 PCA

Se elige PCA porque el dataset tiene seis variables continuas y el método permite resumir la variación en pocos ejes interpretables.

Como los datos ya están centrados y escalados, aplicamos `prcomp()` sin volver a escalar.

```
pca_wholesale <- prcomp(wholesale_log_escalado,  
                        center = FALSE,  
                        scale. = FALSE)  
  
summary(pca_wholesale)
```

Importance of components:

	PC1	PC2	PC3	PC4	PC5	PC6
Standard deviation	1.6262	1.2774	0.8012	0.7786	0.54090	0.42774
Proportion of Variance	0.4408	0.2720	0.1070	0.1010	0.04876	0.03049
Cumulative Proportion	0.4408	0.7127	0.8197	0.9207	0.96951	1.00000

Los dos primeros componentes explican alrededor del 71% de la variación total. Esto indica que el plano PC1-PC2 resume una parte importante de la estructura multivariada.

2.2 Interpretación de los componentes

```
round(pca_wholesale$rotation, 3)
```

	PC1	PC2	PC3	PC4	PC5	PC6
Fresh	-0.105	0.590	-0.640	0.479	-0.040	-0.026
Milk	0.542	0.133	-0.074	-0.062	0.760	0.319
Grocery	0.571	-0.006	-0.133	-0.097	-0.093	-0.799
Frozen	-0.138	0.590	-0.021	-0.792	-0.073	0.005
Detergent	0.551	-0.071	-0.200	-0.083	-0.620	0.510
Delicatessen	0.214	0.530	0.726	0.352	-0.148	0.003

PC1 está asociado principalmente a **Milk**, **Grocery** y **Detergent**. Las tres variables tienen cargas positivas y de magnitud similar.

PC2 está asociado principalmente a **Fresh**, **Frozen** y **Delicatessen**.

Entonces, los dos primeros componentes separan dos patrones principales de consumo: uno más relacionado con **Milk/Grocery/Detergent** y otro más relacionado con **Fresh/Frozen/Delicatessen**.

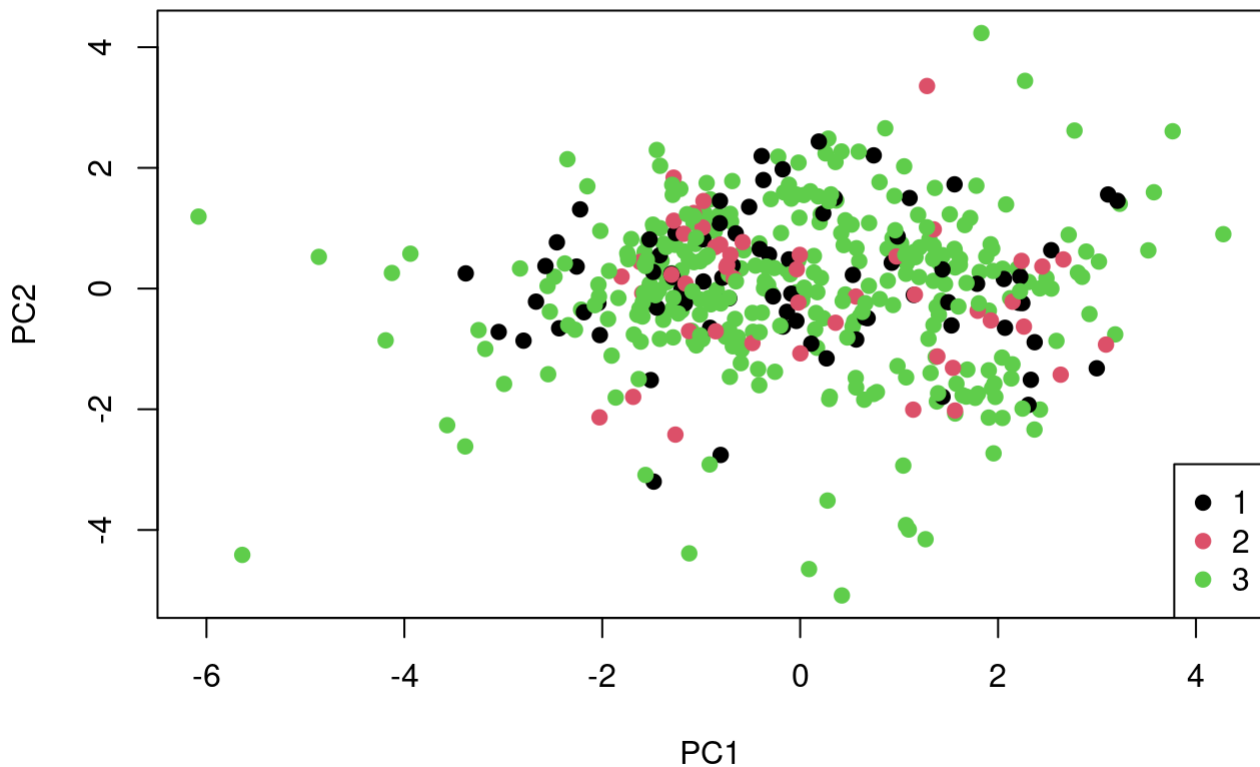
2.3 PCA coloreado por Region

```
scores_pca <- pca_wholesale$x

plot(scores_pca[, 1], scores_pca[, 2],
     col = as.numeric(wholesale_log_escalado_df$Region),
     pch = 19,
     xlab = "PC1",
     ylab = "PC2",
     main = "PCA coloreado por Region")

legend("bottomright",
     legend = levels(wholesale_log_escalado_df$Region),
     col = 1:3,
     pch = 19)
```

PCA coloreado por Region



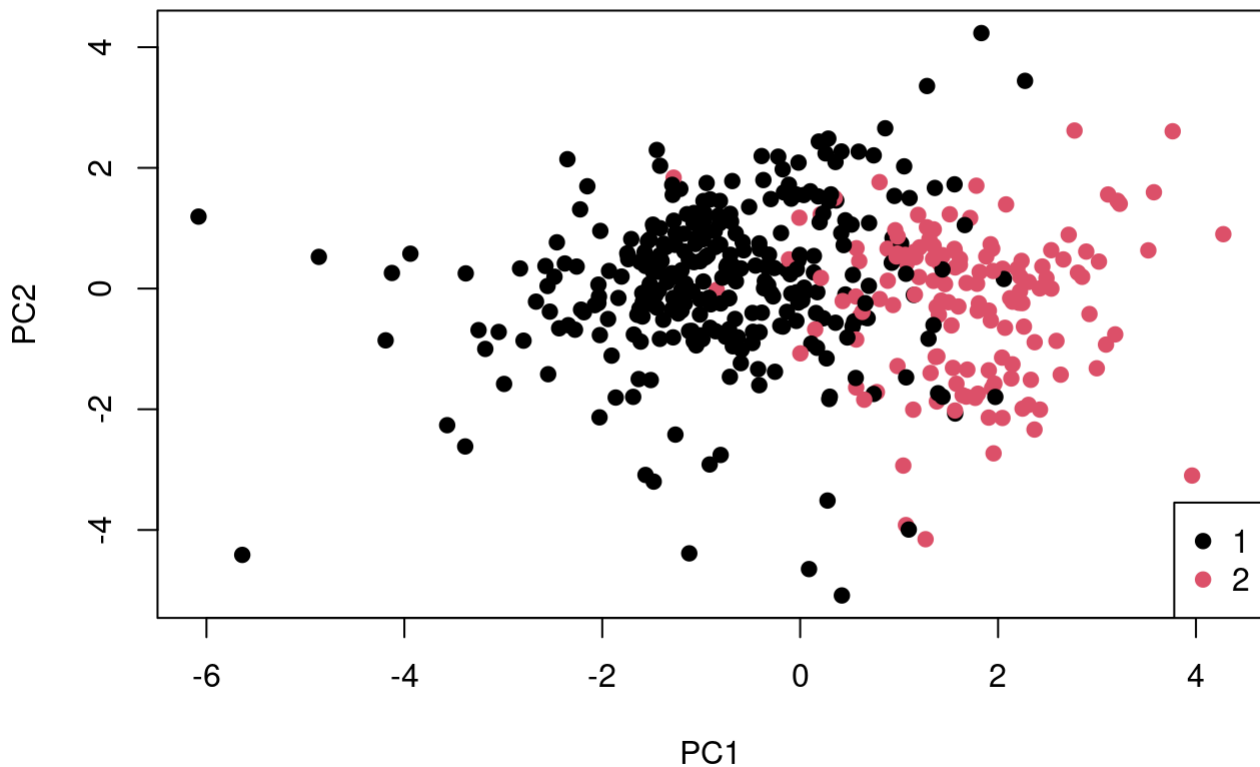
Al colorear por **Region**, los puntos aparecen bastante mezclados. No se observa una separación clara entre las tres regiones.

2.4 PCA coloreado por Channel

```
plot(scores_pca[, 1], scores_pca[, 2],
     col = as.numeric(wholesale_log_escalado_df$Channel),
     pch = 19,
     xlab = "PC1",
     ylab = "PC2",
     main = "PCA coloreado por Channel")

legend("bottomright",
     legend = levels(wholesale_log_escalado_df$Channel),
     col = 1:2,
     pch = 19)
```

PCA coloreado por Channel



Al colorear por [Channel](#), se observa una separación más clara, aunque con solapamiento. El grupo 2 aparece más desplazado hacia valores positivos de PC1. Esto coincide con la importancia de [Milk](#), [Grocery](#) y [Detergent](#) en ese eje.

3. Clustering jerárquico

3.1 Distancias

Calculamos distancia euclidiana y Manhattan para ver si ordenan parecido a los clientes.

```
dist_euc <- dist(wholesale_log_escalado)
dist_man <- dist(wholesale_log_escalado, method = "manhattan")

cor(as.vector(dist_euc),
    as.vector(dist_man),
    method = "spearman")
```

```
[1] 0.9763917
```

La correlación entre ambas distancias es muy alta. Por eso se continúa con distancia euclidiana, que es habitual para datos escalados y coherente con PCA y k-means.

3.2 Comparación de linkage

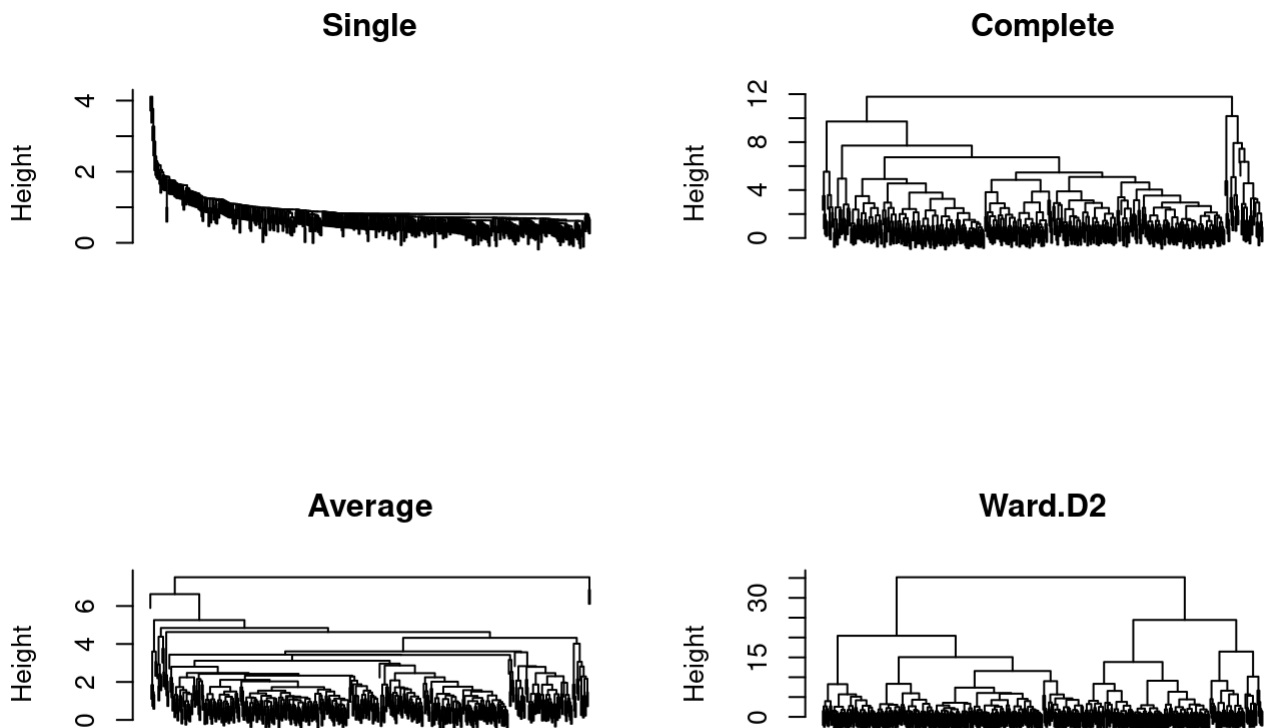
El linkage define cómo se mide la distancia entre grupos ya formados.

```

hc_single   <- hclust(dist_euc, method = "single")
hc_complete <- hclust(dist_euc, method = "complete")
hc_average  <- hclust(dist_euc, method = "average")
hc_ward     <- hclust(dist_euc, method = "ward.D2")

par(mfrow = c(2, 2))
plot(hc_single, main = "Single", sub = "", xlab = "", labels = FALSE)
plot(hc_complete, main = "Complete", sub = "", xlab = "", labels = FALSE)
plot(hc_average, main = "Average", sub = "", xlab = "", labels = FALSE)
plot(hc_ward, main = "Ward.D2", sub = "", xlab = "", labels = FALSE)

```



```

par(mfrow = c(1, 1))

```

Ward.D2 muestra una estructura más clara, con dos grandes ramas principales y grupos más compactos. Por eso se continúa con el clustering jerárquico usando **Ward.D2**.

3.3 Alturas de fusión

```

heights <- sort(hc_ward$height, decreasing = TRUE)

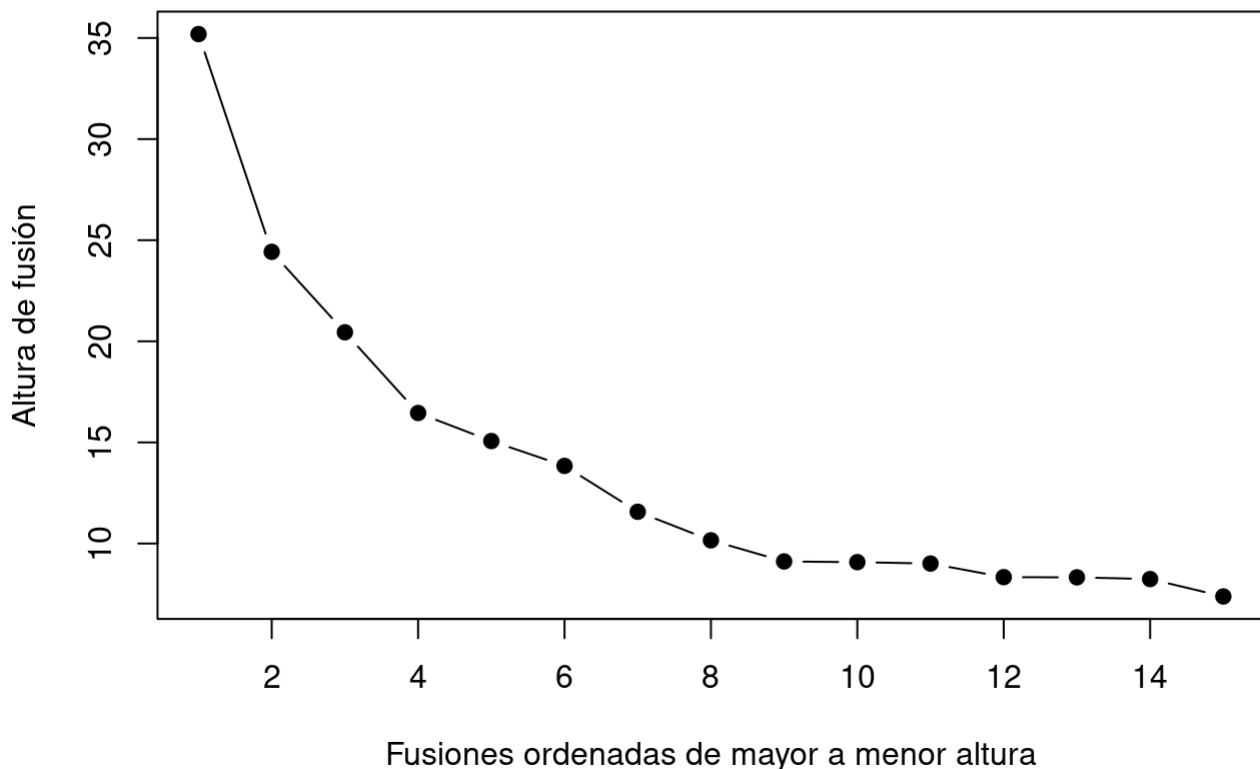
n_max <- 15
plot(1:n_max, heights[1:n_max],
     type = "b",
     pch = 19,
     xlab = "Fusiones ordenadas de mayor a menor altura",

```



```
ylab = "Altura de fusión",  
main = "Scree plot de alturas")
```

Scree plot de alturas

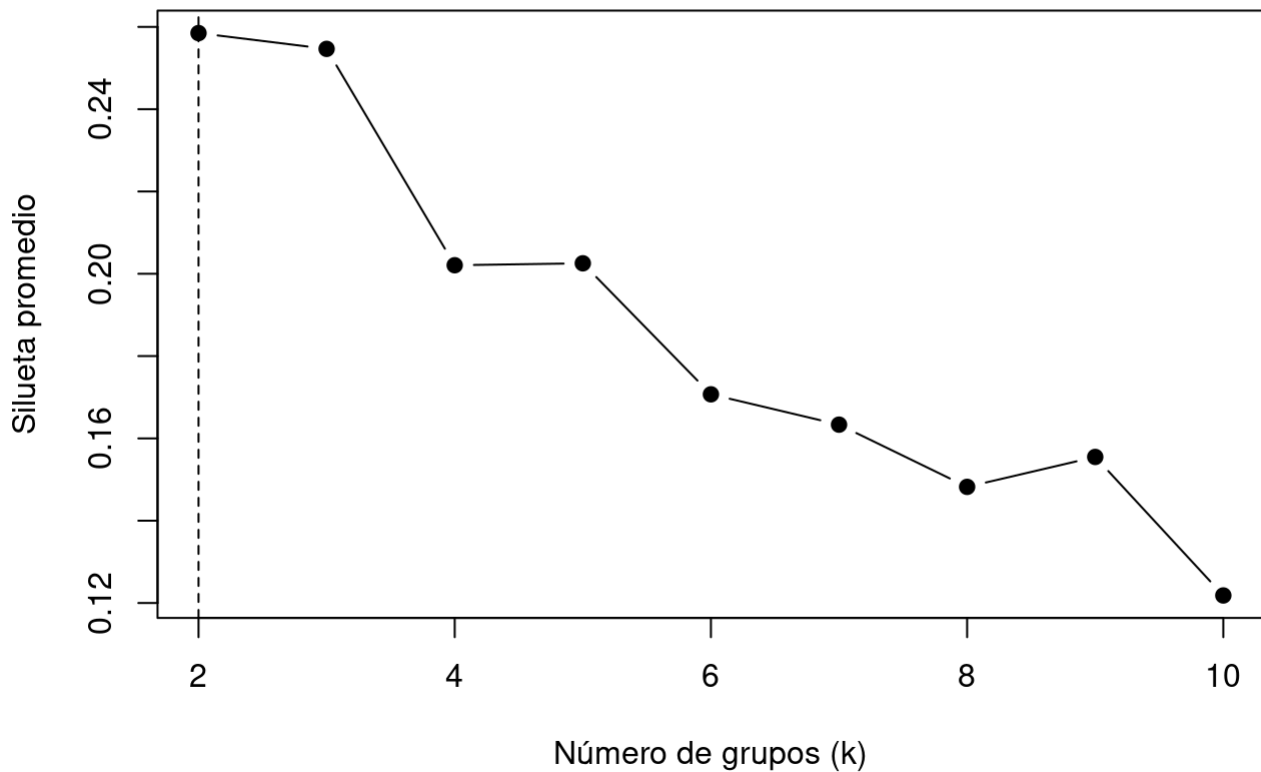


El gráfico de alturas sugiere que el corte podría estar en 2, 3 o 4 grupos. Para decidir mejor, se complementa con silueta promedio.

3.4 Silueta promedio y elección de k

```
sil_ward <- sapply(2:10, function(k) {  
  grupos_temp <- cutree(hc_ward, k = k)  
  sil_temp <- silhouette(grupos_temp, dist_euc)  
  mean(sil_temp[, 3])  
})  
  
plot(2:10, sil_ward,  
     type = "b",  
     pch = 19,  
     xlab = "Número de grupos (k)",  
     ylab = "Silueta promedio",  
     main = "Silueta promedio - Wholesale")  
abline(v = which.max(sil_ward) + 1, lty = 2)
```

Silueta promedio — Wholesale



La silueta promedio más alta aparece en $k = 2$. Por eso se seleccionan dos grupos.

```
grupos <- cutree(hc_ward, k = 2)
table(grupos)
```

```
grupos
  1  2
178 262
```

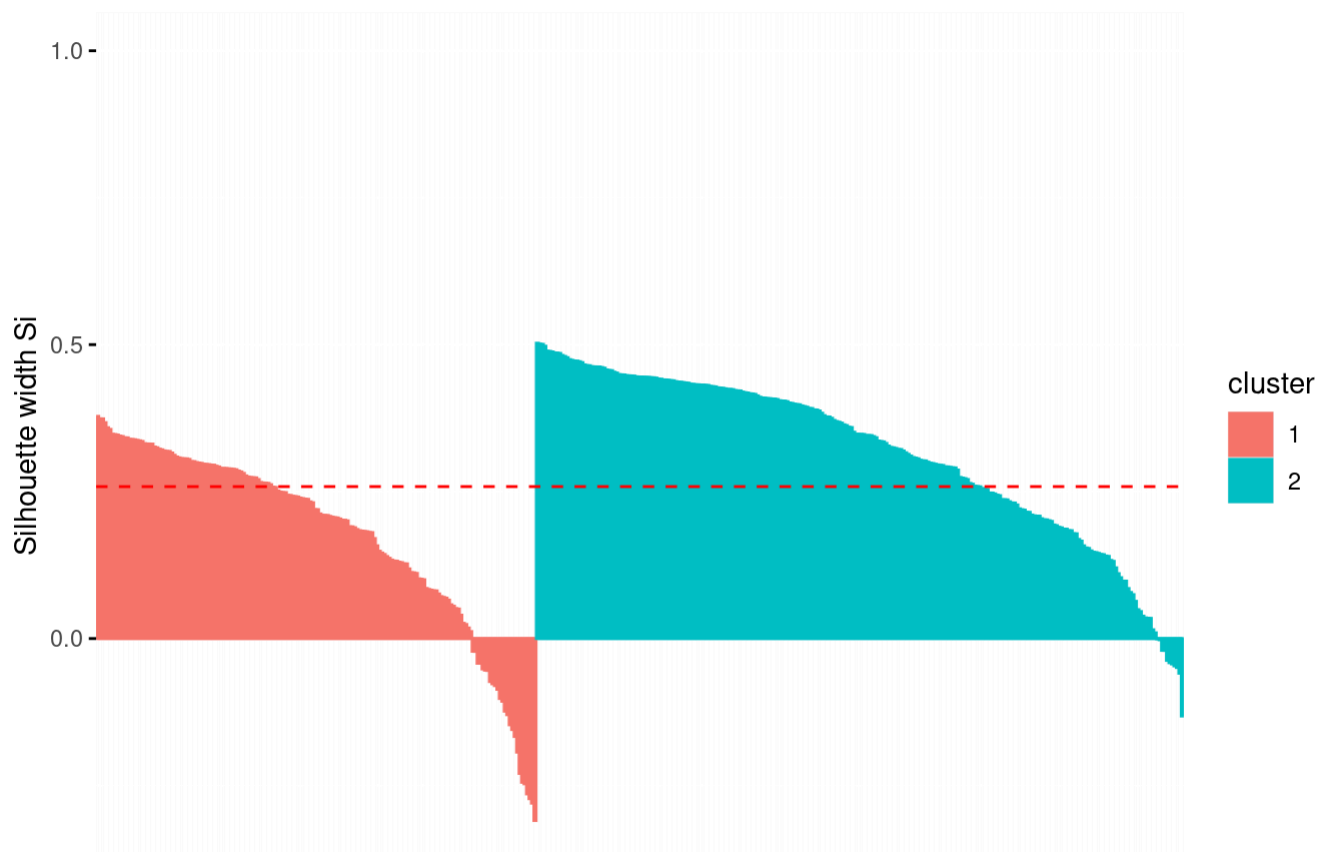
```
sil_2 <- silhouette(grupos, dist_euc)
mean(sil_2[, 3])
```

```
[1] 0.2584953
```

```
fviz_silhouette(sil_2) +
  ggtitle("Silueta por observación para k = 2")
```

	cluster	size	ave.sil.width
1	1	178	0.18
2	2	262	0.31

Silueta por observación para k = 2



La partición es razonable pero no perfecta. La mayoría de las observaciones tienen silueta positiva, aunque algunas aparecen con valores negativos.

3.5 Comparación con Channel

```
tabla <- table(grupos, wholesales$Channel)
tabla
```

```
grupos   1    2
      1  47 131
      2 251  11
```

```
prop.table(tabla, margin = 1)
```

```
grupos      1      2
      1 0.26404494 0.73595506
      2 0.95801527 0.04198473
```

El grupo 1 está compuesto principalmente por observaciones de Channel 2, mientras que el grupo 2 está compuesto casi totalmente por observaciones de Channel 1.

La partición obtenida por clustering jerárquico se relaciona bastante bien con Channel, aunque no de forma perfecta.

4. K-means, GMM, DBSCAN y OPTICS

En este punto se comparan cuatro métodos de clustering. La comparación se hace mirando si los grupos encontrados se relacionan con `Channel`.

4.1 K-means

```
set.seed(123)
km2 <- kmeans(wholesale_log_escalado,
              centers = 2,
              nstart = 25)

table(km2$cluster)
```

```
 1    2
252 188
```

```
VE <- km2$betweenss / km2$totss * 100
VE
```

```
[1] 30.14909
```

K-means separa los datos en dos grupos. La variabilidad explicada es cercana al 30%, lo que indica que la división captura parte de la estructura, aunque no representa una separación perfecta.

```
tabla_km <- table(kmeans = km2$cluster,
                  Channel = wholesale$Channel)

tabla_km
```

```
      Channel
kmeans  1    2
  1 244    8
  2  54 134
```

```
prop.table(tabla_km, margin = 1)
```

```
      Channel
kmeans  1    2
  1 0.96825397 0.03174603
  2 0.28723404 0.71276596
```

K-means recupera bastante bien la estructura asociada a `Channel`: un grupo queda casi completamente asociado a `Channel 1` y el otro principalmente a `Channel 2`.

4.2 GMM con dos grupos

Ajustamos GMM forzado a dos grupos para compararlo directamente con k-means y con `Channel`.

```
gmm2 <- Mclust(wholesale_log_escalado, G = 2)
summary(gmm2)
```

Gaussian finite mixture model fitted by EM algorithm

Mclust VEE (ellipsoidal, equal shape and orientation) model with 2 components:

log-likelihood	n	df	BIC	ICL
-3046.82	440	35	-6306.678	-6373.78

Clustering table:

	1	2
78	362	

```
tabla_gmm2 <- table(GMM = gmm2$classification,
                    Channel = wholesale$Channel)
```

tabla_gmm2

	Channel	
GMM	1	2
1	55	23
2	243	119

```
prop.table(tabla_gmm2, margin = 1)
```

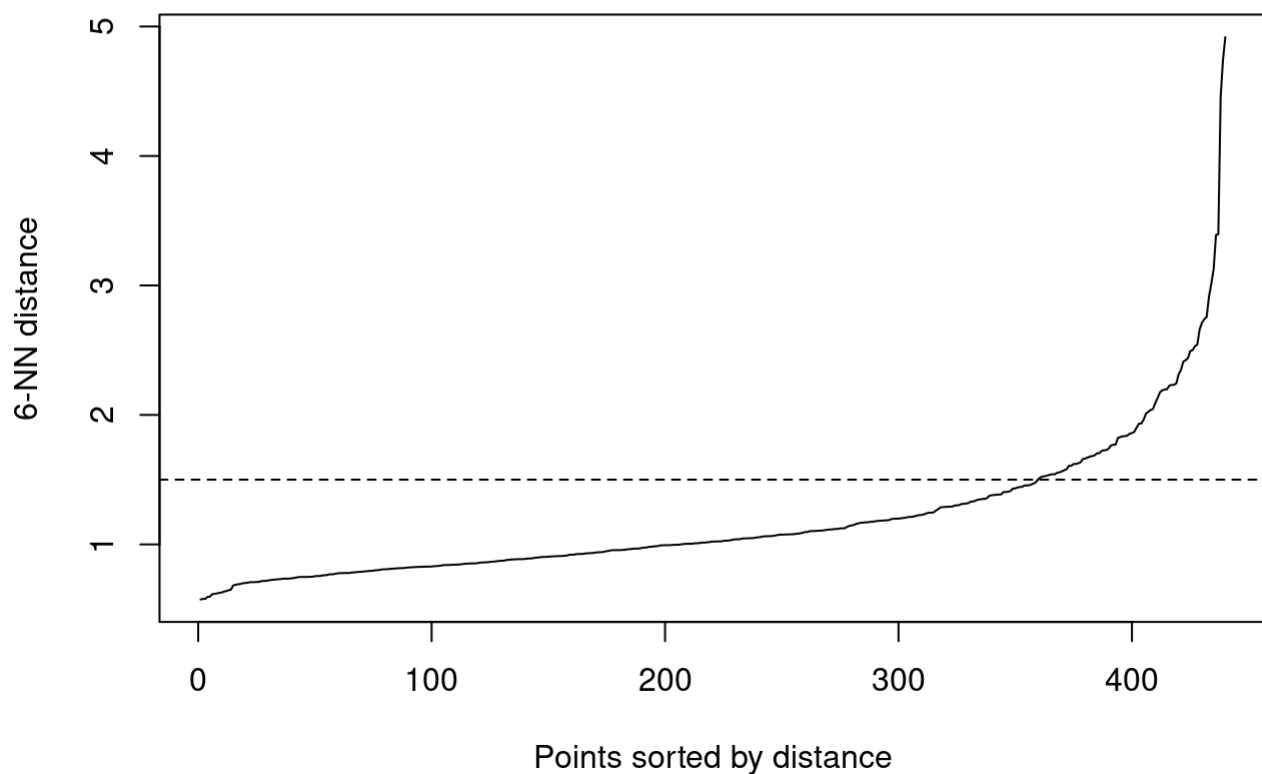
	Channel	
GMM	1	2
1	0.7051282	0.2948718
2	0.6712707	0.3287293

GMM con dos grupos no recupera bien la separación entre **Channel 1** y **Channel 2**. Ambos grupos quedan dominados por **Channel 1**.

4.3 DBSCAN

DBSCAN busca clusters como zonas de alta densidad. Para elegir **eps**, se usa el gráfico de distancia al sexto vecino más cercano, porque el dataset tiene seis variables numéricas.

```
kNNdistplot(wholesale_log_escalado, k = 6)
abline(h = 1.5, lty = 2)
```



Se observa un cambio de pendiente alrededor de $\text{eps} = 1.5$.

```
db <- dbscan(wholesale_log_escalado,
             eps = 1.5,
             minPts = 6)

table(db$cluster)
```

```
0    1
41 399
```

```
tabla_db <- table(DBSCAN = db$cluster,
                  Channel = wholesale$Channel)

tabla_db
```

```
      Channel
DBSCAN  1    2
0      33    8
1     265   134
```

```
prop.table(tabla_db, margin = 1)
```

```
      Channel
DBSCAN  1          2
0 0.8048780 0.1951220
1 0.6641604 0.3358396
```

Con estos parámetros, DBSCAN identifica un único cluster principal y algunas observaciones como ruido. También se probaron otros valores de `eps` entre 1 y 2, pero el comportamiento fue similar.

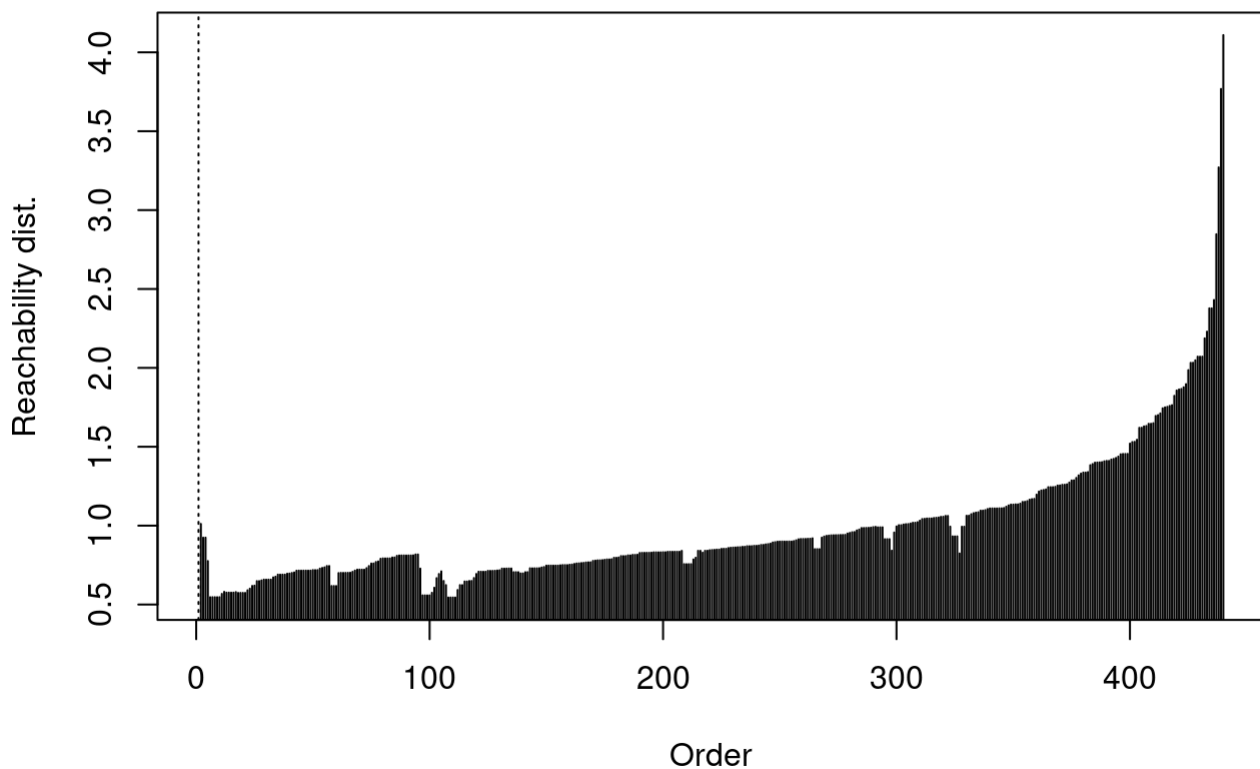
Por eso DBSCAN no resulta adecuado para representar la estructura principal de estos datos.

4.4 OPTICS

OPTICS también trabaja con densidad, pero permite mirar mejor la estructura a través del reachability plot.

```
op <- optics(wholesale_log_escalado,  
             minPts = 6)  
  
plot(op)
```

Reachability Plot



El gráfico no muestra valles grandes y claros. La mayor parte de las observaciones forma una secuencia bastante continua.

```
op_db <- extractDBSCAN(op,  
                       eps_cl = 1.5)  
  
tabla_op <- table(OPTICS = op_db$cluster,  
                 Channel = wholesale$Channel)  
  
tabla_op
```

```

      Channel
OPTICS  1    2
      0  33   8
      1 265 134

```

```
prop.table(tabla_op, margin = 1)
```

```

      Channel
OPTICS      1      2
      0 0.8048780 0.1951220
      1 0.6641604 0.3358396

```

OPTICS identifica un único cluster principal y un conjunto de observaciones como ruido. Tampoco recupera claramente la separación entre canales.

4.5 Comparación general de métodos

De los cuatro métodos comparados:

- k-means fue el único que recuperó claramente la estructura asociada a `Channel`.
- GMM con dos grupos no logró separar los canales.
- DBSCAN y OPTICS formaron un único cluster principal y algunas observaciones como ruido.

Por eso se selecciona k-means con `k = 2` como el método más adecuado para representar la estructura asociada a `Channel`.

5. Clasificación supervisada y semisupervisada

Ahora el objetivo cambia. Ya no buscamos grupos sin etiqueta, sino predecir la clase real `Channel`.

5.1 Separación entrenamiento / prueba

```

X <- as.data.frame(wholesale_log_escalado)
y <- wholesale_factor$Channel
n <- nrow(X)

table(y)

```

```

y
  1    2
298 142

```

```

set.seed(123)
ind_train <- sample(1:n, size = round(0.7 * n))
ind_test  <- setdiff(1:n, ind_train)

X_train <- X[ind_train, ]
X_test  <- X[ind_test, ]

y_train <- y[ind_train]

```



```
y_test <- y[ind_test]
```

```
table(y)
```

```
y
  1  2
298 142
```

```
table(y_train)
```

```
y_train
  1  2
213  95
```

```
table(y_test)
```

```
y_test
  1  2
 85 47
```

Se divide la base en 70% entrenamiento y 30% prueba. La distribución de **Channel** queda parecida en ambos conjuntos.

5.2 LDA

```
datos_train <- data.frame(Channel = y_train, X_train)
datos_test  <- data.frame(Channel = y_test, X_test)

lda_wholesale <- lda(Channel ~ ., data = datos_train)
lda_wholesale
```

Call:

```
lda(Channel ~ ., data = datos_train)
```

Prior probabilities of groups:

```
      1      2
0.6915584 0.3084416
```

Group means:

	Fresh	Milk	Grocery	Frozen	Detergent	Delicatessen
1	0.1270009	-0.3773885	-0.4102947	0.1920311	-0.4980623	-0.1329240
2	-0.2895862	0.8638556	1.0076350	-0.3422680	1.0648857	0.1897584

Coefficients of linear discriminants:

	LD1
Fresh	-0.12832113
Milk	0.23204324
Grocery	0.35122895
Frozen	-0.14344998
Detergent	0.94445625
Delicatessen	0.01405003

LDA calcula una función discriminante lineal porque **Channel** tiene solo dos grupos. Las medias por grupo muestran que el canal 1 tiene valores más altos en **Fresh** y **Frozen**, mientras que el canal 2 tiene valores más altos en **Milk**, **Grocery** y **Detergent**.

```
pred_lda <- predict(lda_wholesale,
                    newdata = datos_test)

tabla_lda <- table(real = y_test,
                  predicho = pred_lda$class)

tabla_lda
```

```
      predicho
real  1  2
     1 80  5
     2  7 40
```

```
accuracy_lda <- sum(diag(tabla_lda)) / sum(tabla_lda)
accuracy_lda
```

```
[1] 0.9090909
```

```
acierto_por_clase_lda <- diag(prop.table(tabla_lda, margin = 1))
acierto_por_clase_lda
```

```
      1      2
0.9411765 0.8510638
```

```
balanced_accuracy_lda <- mean(acierto_por_clase_lda)
balanced_accuracy_lda
```

```
[1] 0.8961202
```

La accuracy global indica la proporción total de aciertos. Pero como las clases no están completamente balanceadas, también se mira el acierto por clase y la precisión balanceada.

5.3 EDDA

```
edda_wholesale <- MclustDA(X_train,
                          class = y_train,
                          modelType = "EDDA")

summary(edda_wholesale)
```

```
-----
Gaussian finite mixture model for classification
-----
```

```
EDDA
```

```
-----
log-likelihood   n df      BIC
```

-2228.09 308 40 -4685.383

Classes	n	%	Model	G
1	213	69.16	VVE	1
2	95	30.84	VVE	1

Training confusion matrix

```
-----
      Predicted
Classes  1    2
      1 194   19
      2   9   86
```

Classification error = 0.0909

Brier score = 0.0728

```
pred_edda <- predict(edda_wholesale,
                     newdata = X_test)

tabla_edda <- table(real = y_test,
                   predicho = pred_edda$classification)

tabla_edda
```

```
      predicho
real  1    2
      1  76   9
      2   4  43
```

```
accuracy_edda <- sum(diag(tabla_edda)) / sum(tabla_edda)
accuracy_edda
```

[1] 0.9015152

```
acierto_por_clase_edda <- diag(prop.table(tabla_edda, margin = 1))
acierto_por_clase_edda
```

```
      1      2
0.8941176 0.9148936
```

```
balanced_accuracy_edda <- mean(acierto_por_clase_edda)
balanced_accuracy_edda
```

[1] 0.9045056

EDDA ajusta un modelo supervisado basado en mezclas gaussianas. Para comparar con LDA, se evalúa sobre el conjunto de prueba.

5.4 Mixtura gaussiana semisupervisada

En un planteo semisupervisado, una parte de las etiquetas se conoce y otra parte se oculta. El modelo usa las etiquetas conocidas como guía, pero también aprovecha la estructura multivariada de las

observaciones sin etiqueta.

```
set.seed(123)

y_semi <- y_train
n_train <- nrow(X_train)
ind_sin_etiqueta <- sample(1:n_train,
                           size = round(n_train / 2))

y_semi[ind_sin_etiqueta] <- NA

table(y_train)
```

```
y_train
  1   2
213 95
```

```
table(y_semi, useNA = "ifany")
```

```
y_semi
  1   2 <NA>
102  52 154
```

```
semi_wholesale <- MclustSSC(X_train,
                           class = y_semi)

summary(semi_wholesale)
```

Gaussian finite mixture model for semi-supervised classification

log-likelihood	n	df	BIC
-2245.504	308	40	-4720.212

Classes	n	%	Model	G
1	102	33.12	VVE	1
2	52	16.88	VVE	1
<NA>	154	50.00		

Classification summary

	Predicted	
Classes	1	2
1	102	0
2	0	52
<NA>	103	51

Se oculta el 50% de las etiquetas dentro del conjunto de entrenamiento. El 30% de prueba queda fuera del ajuste y se usa para evaluar.

```
pred_semi <- predict(semi_wholesale,
                    newdata = X_test)
```

```
tabla_semi <- table(real = y_test,
                    predicho = pred_semi$classification)
```

```
tabla_semi
```

```
      predicho
real  1  2
  1  77  8
  2   2 45
```

```
accuracy_semi <- sum(diag(tabla_semi)) / sum(tabla_semi)
accuracy_semi
```

```
[1] 0.9242424
```

```
acierto_por_clase_semi <- diag(prop.table(tabla_semi, margin = 1))
acierto_por_clase_semi
```

```
      1      2
0.9058824 0.9574468
```

```
balanced_accuracy_semi <- mean(acierto_por_clase_semi)
balanced_accuracy_semi
```

```
[1] 0.9316646
```

5.5 Comparación de modelos

```
comparacion_clasificacion <- data.frame(
  Modelo = c("LDA", "EDDA", "Semi-S"),
  Accuracy = c(accuracy_lda,
               accuracy_edda,
               accuracy_semi),
  Ch_1 = c(acierto_por_clase_lda[1],
            acierto_por_clase_edda[1],
            acierto_por_clase_semi[1]),
  Ch_2 = c(acierto_por_clase_lda[2],
            acierto_por_clase_edda[2],
            acierto_por_clase_semi[2]),
  B_Accuracy = c(balanced_accuracy_lda,
                 balanced_accuracy_edda,
                 balanced_accuracy_semi)
)
```

```
comparacion_clasificacion
```

```
      Modelo Accuracy      Ch_1      Ch_2 B_Accuracy
1      LDA 0.9090909 0.9411765 0.8510638 0.8961202
2      EDDA 0.9015152 0.8941176 0.9148936 0.9045056
3 Semi-S 0.9242424 0.9058824 0.9574468 0.9316646
```

En la base completa, el modelo semisupervisado obtiene el mejor rendimiento sobre el conjunto de prueba. Por eso se selecciona como clasificador principal en este análisis.

5.6 Gráfico del modelo seleccionado

El gráfico se realiza en el plano PCA para visualizar el conjunto de prueba. Los puntos están coloreados por **Channel** real, y los errores se marcan con otro símbolo.

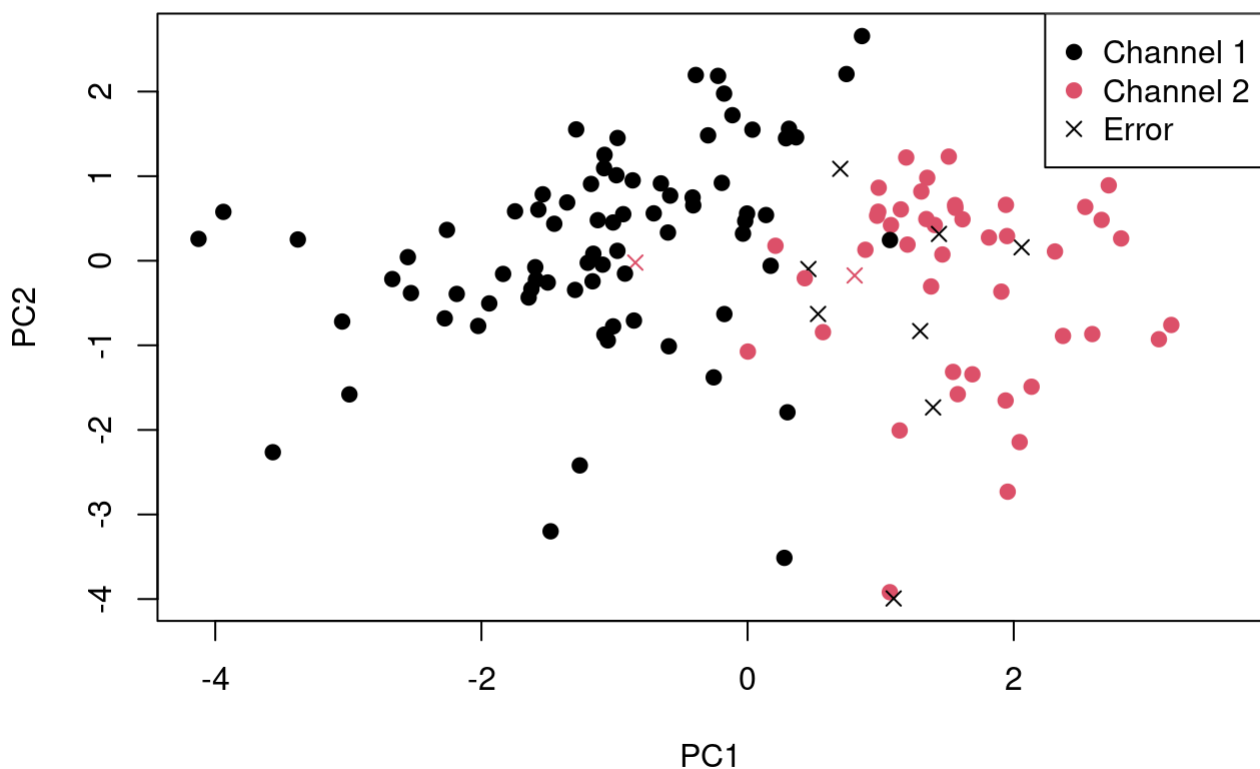
```
scores_pca_df <- as.data.frame(pca_wholesale$x)
scores_test <- scores_pca_df[ind_test, ]

scores_test$Channel_real <- y_test
scores_test$Channel_pred <- pred_semi$classification
scores_test$Clasificacion <- ifelse(scores_test$Channel_real == scores_test$Channel_pred,
                                   "Correcta", "Error")

plot(scores_test$PC1, scores_test$PC2,
     col = as.numeric(scores_test$Channel_real),
     pch = ifelse(scores_test$Clasificacion == "Correcta", 19, 4),
     xlab = "PC1",
     ylab = "PC2",
     main = "Modelo semisupervisado sobre conjunto de prueba")

legend("topright",
     legend = c("Channel 1", "Channel 2", "Error"),
     col = c(1, 2, 1),
     pch = c(19, 19, 4))
```

Modelo semisupervisado sobre conjunto de prueba



6. Análisis de sensibilidad: base sin outliers

En la entrega original se transformaron los datos, pero no se repitieron los análisis quitando los posibles outliers multivariados. Esta sección agrega esa mejora.

La idea no es afirmar que esos datos sean errores, sino revisar si las conclusiones principales cambian cuando se excluyen las observaciones más alejadas.

6.1 Construcción de la base sin outliers

```
wholesale_sin_out <- wholesale[!outliers, ]
wholesale_num_sin_out <- wholesale_num[!outliers, ]
wholesale_factor_sin_out <- wholesale_factor[!outliers, ]

wholesale_log_sin_out <- log1p(wholesale_num_sin_out)
wholesale_log_escalado_sin_out <- scale(wholesale_log_sin_out,
                                       center = TRUE,
                                       scale = TRUE)

nrow(wholesale)
```

```
[1] 440
```

```
sum(outliers)
```

```
[1] 11
```

```
nrow(wholesale_sin_out)
```

```
[1] 429
```

```
table(wholesale_factor$Channel)
```

```
 1    2
298 142
```

```
table(wholesale_factor_sin_out$Channel)
```

```
 1    2
288 141
```

Se quitan 11 observaciones. La distribución de **Channel** casi no cambia: la mayoría de los outliers pertenecían a **Channel 1**.

6.2 PCA completo vs sin outliers

```
pca_wholesale_sin_out <- prcomp(wholesale_log_escalado_sin_out,
                                center = FALSE,
                                scale. = FALSE)

var_pca_full <- (pca_wholesale$sdev^2) / sum(pca_wholesale$sdev^2)
var_pca_sin_out <- (pca_wholesale_sin_out$sdev^2) /
  sum(pca_wholesale_sin_out$sdev^2)

comparacion_pca <- data.frame(
  Conjunto = c("Completo", "Sin outliers"),
  PC1 = c(var_pca_full[1], var_pca_sin_out[1]),
  PC2 = c(var_pca_full[2], var_pca_sin_out[2]),
  PC1_PC2 = c(sum(var_pca_full[1:2]),
               sum(var_pca_sin_out[1:2]))
)

comparacion_pca
```

	Conjunto	PC1	PC2	PC1_PC2
1	Completo	0.4407775	0.2719492	0.7127267
2	Sin outliers	0.4573575	0.2731607	0.7305182

```
round(pca_wholesale$rotation, 3)
```

	PC1	PC2	PC3	PC4	PC5	PC6
Fresh	-0.105	0.590	-0.640	0.479	-0.040	-0.026
Milk	0.542	0.133	-0.074	-0.062	0.760	0.319
Grocery	0.571	-0.006	-0.133	-0.097	-0.093	-0.799
Frozen	-0.138	0.590	-0.021	-0.792	-0.073	0.005
Detergent	0.551	-0.071	-0.200	-0.083	-0.620	0.510
Delicatessen	0.214	0.530	0.726	0.352	-0.148	0.003

```
round(pca_wholesale_sin_out$rotation, 3)
```

	PC1	PC2	PC3	PC4	PC5	PC6
Fresh	-0.145	0.585	-0.506	-0.616	-0.021	-0.028
Milk	0.534	0.157	-0.045	0.045	0.788	-0.255
Grocery	0.567	0.030	-0.179	0.011	-0.146	0.790
Frozen	-0.166	0.580	-0.214	0.766	-0.050	0.029
Detergent	0.546	-0.066	-0.273	0.077	-0.556	-0.555
Delicatessen	0.216	0.540	0.768	-0.161	-0.213	-0.038

Al quitar los outliers, el PCA casi no cambia. Los dos primeros componentes pasan de explicar alrededor del 71.3% a 73.1%. PC1 sigue asociado a **Milk**, **Grocery** y **Detergent**, y PC2 sigue asociado a **Fresh**, **Frozen** y **Delicatessen**.

```
scores_pca_sin_out <- pca_wholesale_sin_out$x

plot(scores_pca_sin_out[, 1], scores_pca_sin_out[, 2],
     col = as.numeric(wholesale_factor_sin_out$Channel),
     pch = 19,
     xlab = "PC1",
     ylab = "PC2",
     main = "PCA sin outliers coloreado por Channel")
```

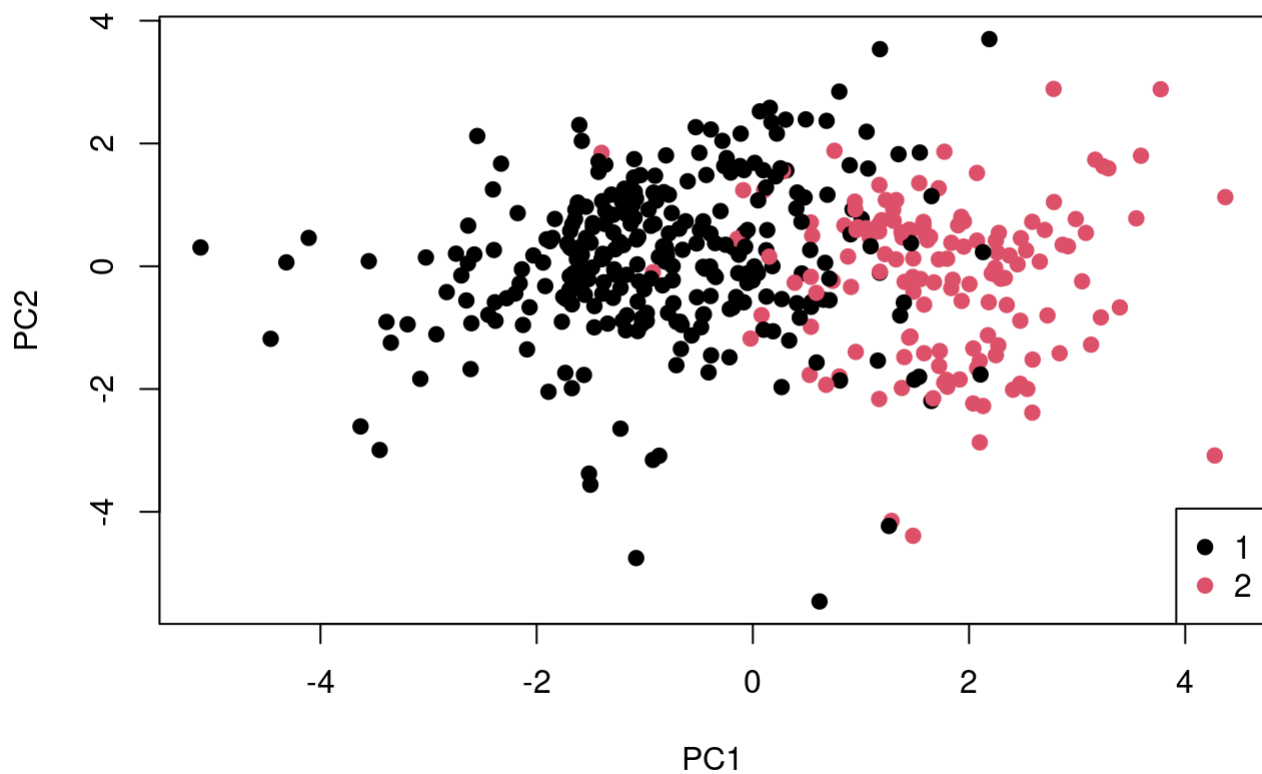


```

legend("bottomright",
      legend = levels(wholesale_factor_sin_out$Channel),
      col = 1:2,
      pch = 19)

```

PCA sin outliers coloreado por Channel



6.3 Clustering jerárquico sin outliers

```

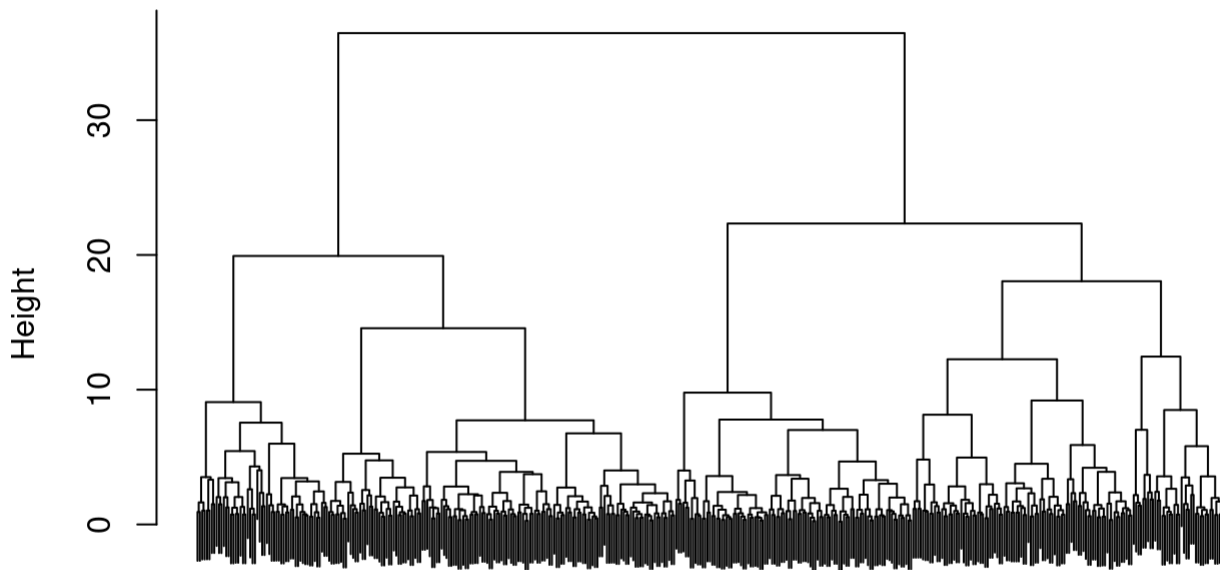
dist_euc_sin_out <- dist(wholesale_log_escalado_sin_out)

hc_ward_sin_out <- hclust(dist_euc_sin_out,
                          method = "ward.D2")

plot(hc_ward_sin_out,
     main = "Ward.D2 sin outliers",
     sub = "",
     xlab = "",
     labels = FALSE)

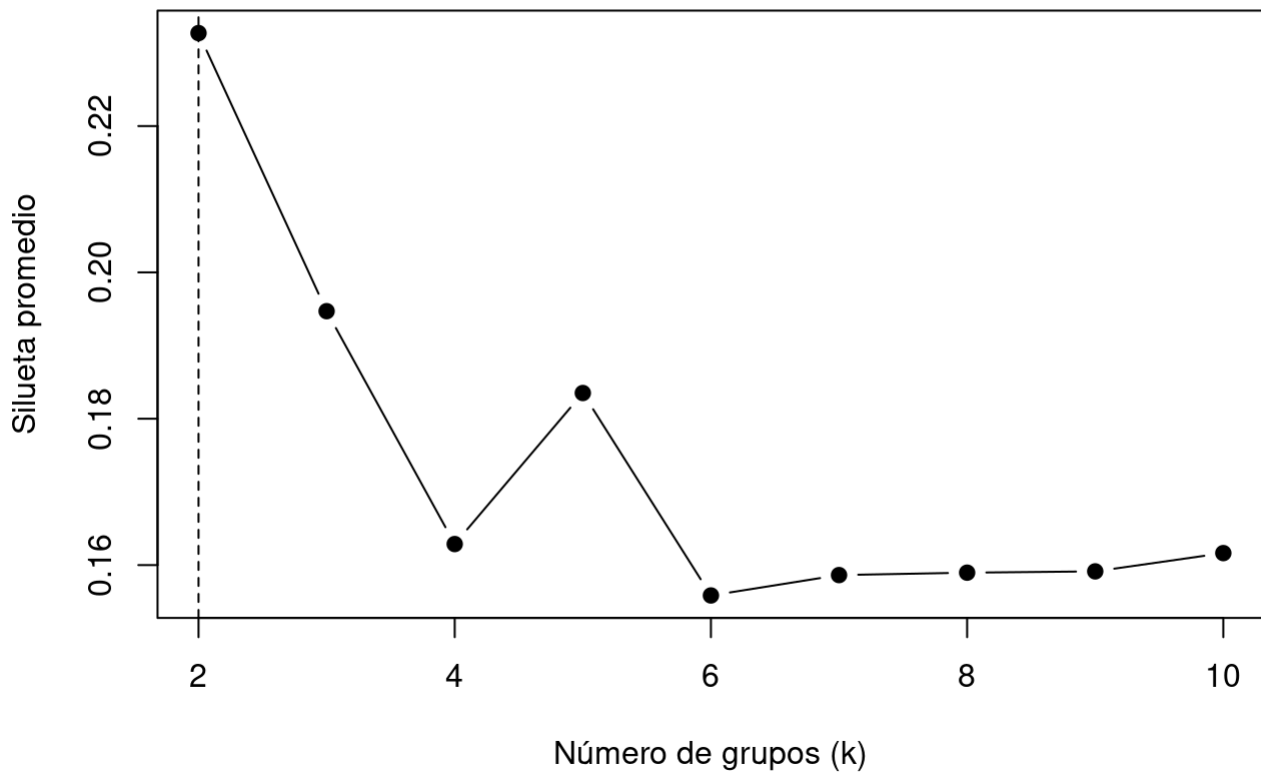
```

Ward.D2 sin outliers



```
sil_ward_sin_out <- sapply(2:10, function(k) {  
  grupos_temp <- cutree(hc_ward_sin_out, k = k)  
  sil_temp <- silhouette(grupos_temp, dist_euc_sin_out)  
  mean(sil_temp[, 3])  
})  
  
plot(2:10, sil_ward_sin_out,  
     type = "b",  
     pch = 19,  
     xlab = "Número de grupos (k)",  
     ylab = "Silueta promedio",  
     main = "Silueta promedio sin outliers")  
abline(v = which.max(sil_ward_sin_out) + 1, lty = 2)
```

Silueta promedio sin outliers



```
grupos_sin_out <- cutree(hc_ward_sin_out, k = 2)

tabla_hc_sin_out <- table(Grupo = grupos_sin_out,
                          Channel = wholesale_factor_sin_out$Channel)

tabla_hc_sin_out
```

```
      Channel
Grupo  1    2
  1  95 134
  2 193   7
```

```
prop.table(tabla_hc_sin_out, margin = 1)
```

```
      Channel
Grupo      1      2
  1 0.4148472 0.5851528
  2 0.9650000 0.0350000
```

Al quitar los outliers, el clustering jerárquico mantiene un grupo muy asociado a **Channel 1**, pero el grupo asociado a **Channel 2** queda más mezclado. La estructura sigue relacionada con **Channel**, aunque con menor claridad que en la base completa.

6.4 K-means sin outliers

```
set.seed(123)
km2_sin_out <- kmeans(wholesale_log_escalado_sin_out,
                      centers = 2,
                      nstart = 25)

table(km2_sin_out$cluster)
```

```
 1  2
179 250
```

```
VE_sin_out <- km2_sin_out$betweenss / km2_sin_out$totss * 100
VE_sin_out
```

```
[1] 32.20307
```

```
tabla_km_sin_out <- table(kmeans = km2_sin_out$cluster,
                          Channel = wholesale_factor_sin_out$Channel)

tabla_km_sin_out
```

```
      Channel
kmeans  1    2
 1   47 132
 2  241   9
```

```
prop.table(tabla_km_sin_out, margin = 1)
```

```
      Channel
kmeans      1      2
 1 0.2625698 0.7374302
 2 0.9640000 0.0360000
```

Al quitar los outliers, k-means sigue recuperando claramente la estructura asociada a **Channel**. La variabilidad explicada aumenta de alrededor del 30.1% a 32.2%, lo que indica una partición algo más compacta.

6.5 GMM sin outliers

```
gmm2_sin_out <- Mclust(wholesale_log_escalado_sin_out,
                       G = 2)

summary(gmm2_sin_out)
```

```
-----
Gaussian finite mixture model fitted by EM algorithm
-----
```

Mclust VVE (ellipsoidal, equal orientation) model with 2 components:

```
log-likelihood   n df      BIC      ICL
```

-2922.807 429 40 -6088.072 -6132.017

Clustering table:

	1	2
169	260	

```
tabla_gmm2_sin_out <- table(GMM = gmm2_sin_out$classification,  
                             Channel = wholesale_factor_sin_out$Channel)
```

tabla_gmm2_sin_out

	Channel	
GMM	1	2
1	37	132
2	251	9

```
prop.table(tabla_gmm2_sin_out, margin = 1)
```

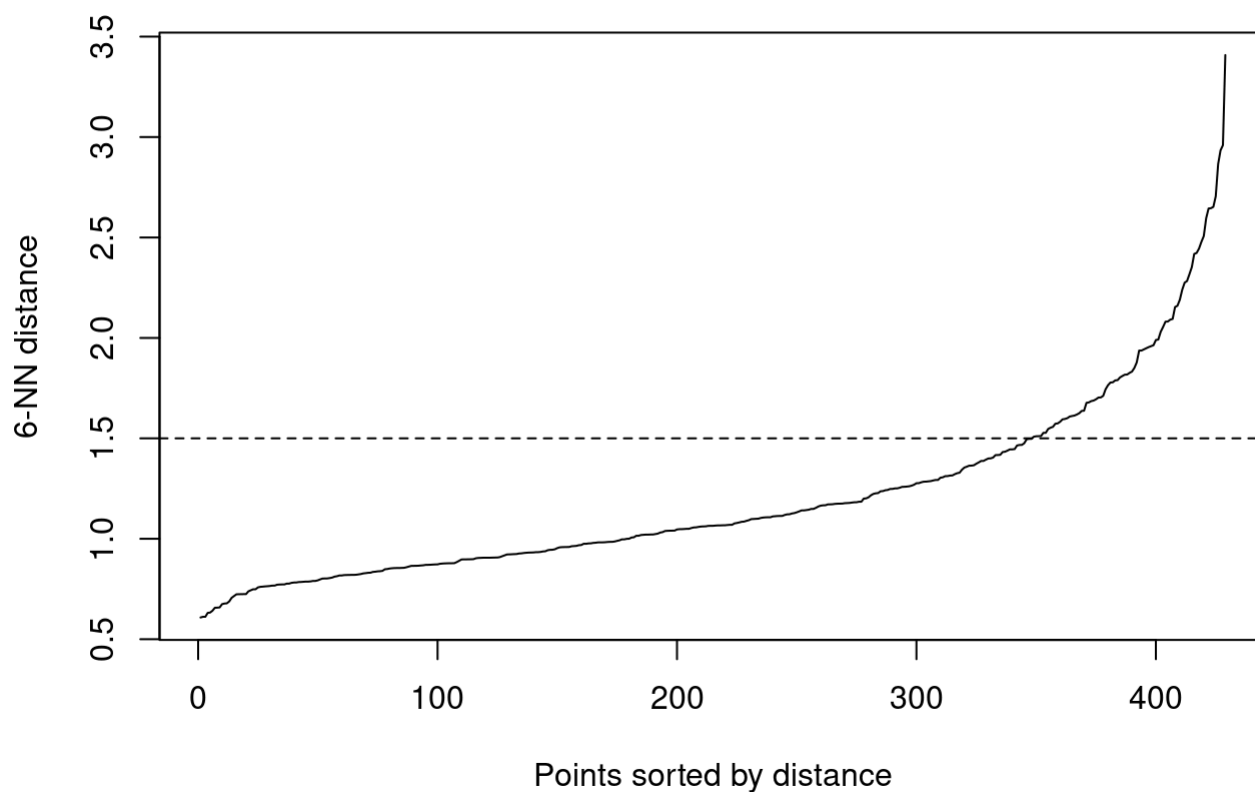
	Channel	
GMM	1	2
1	0.21893491	0.78106509
2	0.96538462	0.03461538

GMM mejora mucho al quitar los outliers. En la base completa, GMM con dos grupos no separaba bien **Channel**. Sin outliers, forma un grupo principalmente asociado a **Channel 2** y otro casi completamente asociado a **Channel 1**.

Esto indica que los outliers afectaban bastante al modelo gaussiano.

6.6 DBSCAN y OPTICS sin outliers

```
kNNdistplot(wholesale_log_escalado_sin_out, k = 6)  
abline(h = 1.5, lty = 2)
```



```
db_sin_out <- dbscan(wholesale_log_escalado_sin_out,
                     eps = 1.5,
                     minPts = 6)

tabla_db_sin_out <- table(DBSCAN = db_sin_out$cluster,
                          Channel = wholesale_factor_sin_out$Channel)

tabla_db_sin_out
```

	Channel	
DBSCAN	1	2
0	28	8
1	260	133

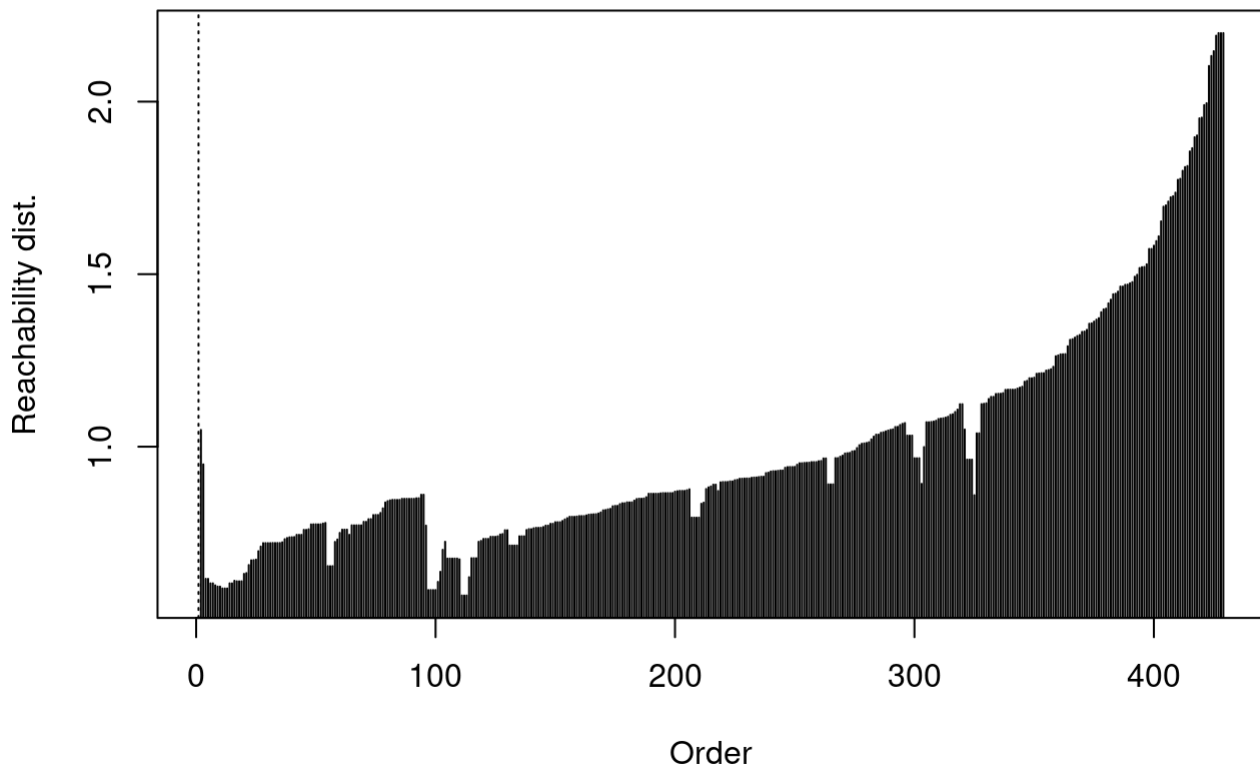
```
prop.table(tabla_db_sin_out, margin = 1)
```

	Channel	
DBSCAN	1	2
0	0.7777778	0.2222222
1	0.6615776	0.3384224

```
op_sin_out <- optics(wholesale_log_escalado_sin_out,
                     minPts = 6)

plot(op_sin_out)
```

Reachability Plot



```
op_db_sin_out <- extractDBSCAN(op_sin_out,  
                               eps_cl = 1.5)  
  
tabla_op_sin_out <- table(OPTICS = op_db_sin_out$cluster,  
                          Channel = wholesale_factor_sin_out$Channel)  
  
tabla_op_sin_out
```

```
      Channel  
OPTICS   1   2  
0      28   8  
1     260 133
```

```
prop.table(tabla_op_sin_out, margin = 1)
```

```
      Channel  
OPTICS      1      2  
0 0.7777778 0.2222222  
1 0.6615776 0.3384224
```

DBSCAN y OPTICS casi no cambian sin outliers. Siguen formando un único cluster principal y algunas observaciones como ruido. Entonces, no recuperan la separación entre canales. Esto sugiere que el problema no eran solo los outliers, sino que los datos no presentan clusters densos claramente separados.

6.7 Clasificación sin outliers

Repetimos la comparación entre LDA, EDDA y modelo semisupervisado usando la base sin outliers.

```
X_sin_out <- as.data.frame(wholesale_log_escalado_sin_out)
y_sin_out <- wholesale_factor_sin_out$Channel

set.seed(123)
n_sin_out <- nrow(X_sin_out)
ind_train_sin_out <- sample(1:n_sin_out,
                           size = round(0.7 * n_sin_out))
ind_test_sin_out <- setdiff(1:n_sin_out, ind_train_sin_out)

X_train_sin_out <- X_sin_out[ind_train_sin_out, ]
X_test_sin_out <- X_sin_out[ind_test_sin_out, ]

y_train_sin_out <- y_sin_out[ind_train_sin_out]
y_test_sin_out <- y_sin_out[ind_test_sin_out]

table(y_sin_out)
```

```
y_sin_out
  1  2
288 141
```

```
table(y_train_sin_out)
```

```
y_train_sin_out
  1  2
203  97
```

```
table(y_test_sin_out)
```

```
y_test_sin_out
  1  2
85 44
```

LDA sin outliers

```
datos_train_sin_out <- data.frame(Channel = y_train_sin_out,
                                   X_train_sin_out)
datos_test_sin_out <- data.frame(Channel = y_test_sin_out,
                                  X_test_sin_out)

lda_sin_out <- lda(Channel ~ ., data = datos_train_sin_out)
pred_lda_sin_out <- predict(lda_sin_out,
                            newdata = datos_test_sin_out)

tabla_lda_sin_out <- table(real = y_test_sin_out,
                           predicho = pred_lda_sin_out$class)

tabla_lda_sin_out
```

```
      predicho
real  1  2
```



```
1 81 4
2 5 39
```

```
accuracy_lda_sin_out <- sum(diag(tabla_lda_sin_out)) /
  sum(tabla_lda_sin_out)

acierto_por_clase_lda_sin_out <- diag(prop.table(tabla_lda_sin_out,
  margin = 1))

balanced_accuracy_lda_sin_out <- mean(acierto_por_clase_lda_sin_out)
```

EDDA sin outliers

```
edda_sin_out <- MclustDA(X_train_sin_out,
  class = y_train_sin_out,
  modelType = "EDDA")

pred_edda_sin_out <- predict(edda_sin_out,
  newdata = X_test_sin_out)

tabla_edda_sin_out <- table(real = y_test_sin_out,
  predicho = pred_edda_sin_out$classification)

tabla_edda_sin_out
```

```
      predicho
real 1 2
1 79 6
2 6 38
```

```
accuracy_edda_sin_out <- sum(diag(tabla_edda_sin_out)) /
  sum(tabla_edda_sin_out)

acierto_por_clase_edda_sin_out <- diag(prop.table(tabla_edda_sin_out,
  margin = 1))

balanced_accuracy_edda_sin_out <- mean(acierto_por_clase_edda_sin_out)
```

Semisupervisado sin outliers

```
set.seed(123)
y_semi_sin_out <- y_train_sin_out

n_train_sin_out <- nrow(X_train_sin_out)
ind_sin_etiqueta_sin_out <- sample(1:n_train_sin_out,
  size = round(n_train_sin_out / 2))

y_semi_sin_out[ind_sin_etiqueta_sin_out] <- NA

table(y_train_sin_out)
```

```
y_train_sin_out
1 2
```

```
table(y_semi_sin_out, useNA = "ifany")
```

```
y_semi_sin_out
  1    2 <NA>
103  47 150
```

```
semi_sin_out <- MclustSSC(X_train_sin_out,
                        class = y_semi_sin_out)

pred_semi_sin_out <- predict(semi_sin_out,
                            newdata = X_test_sin_out)

tabla_semi_sin_out <- table(real = y_test_sin_out,
                           predicho = pred_semi_sin_out$classification)

tabla_semi_sin_out
```

```
      predicho
real  1  2
  1 80  5
  2  7 37
```

```
accuracy_semi_sin_out <- sum(diag(tabla_semi_sin_out)) /
  sum(tabla_semi_sin_out)

acierto_por_clase_semi_sin_out <- diag(prop.table(tabla_semi_sin_out,
                                                  margin = 1))

balanced_accuracy_semi_sin_out <- mean(acierto_por_clase_semi_sin_out)
```

Comparación final sin outliers

```
comparacion_clasificacion_sin_out <- data.frame(
  Modelo = c("LDA", "EDDA", "Semi-S"),
  Accuracy = c(accuracy_lda_sin_out,
               accuracy_edda_sin_out,
               accuracy_semi_sin_out),
  Ch_1 = c(acierto_por_clase_lda_sin_out[1],
            acierto_por_clase_edda_sin_out[1],
            acierto_por_clase_semi_sin_out[1]),
  Ch_2 = c(acierto_por_clase_lda_sin_out[2],
            acierto_por_clase_edda_sin_out[2],
            acierto_por_clase_semi_sin_out[2]),
  B_Accuracy = c(balanced_accuracy_lda_sin_out,
                 balanced_accuracy_edda_sin_out,
                 balanced_accuracy_semi_sin_out)
)

comparacion_clasificacion_sin_out
```

	Modelo	Accuracy	Ch_1	Ch_2	B_Accuracy
1	LDA	0.9302326	0.9529412	0.8863636	0.9196524
2	EDDA	0.9069767	0.9294118	0.8636364	0.8965241
3	Semi-S	0.9069767	0.9411765	0.8409091	0.8910428

En la base sin outliers, LDA pasa a ser el mejor modelo. Tiene la mayor accuracy global y la mayor precisión balanceada. EDDA y el modelo semisupervisado también funcionan bien, pero quedan un poco por debajo.

Esto sugiere que, al quitar los casos extremos, la separación entre canales se vuelve más simple y un modelo lineal alcanza para clasificar bastante bien.

Cierre general

El análisis muestra que **Channel1** está bastante relacionado con los patrones de gasto de los clientes. Esta relación aparece desde la exploración inicial, se ve en el PCA y también aparece en algunos métodos de clustering.

En la base completa, k-means fue el método no supervisado que mejor recuperó la estructura asociada a **Channel1**. En clasificación, el modelo semisupervisado obtuvo el mejor rendimiento.

Al repetir el análisis sin outliers, las conclusiones principales se mantienen en PCA y k-means. Sin embargo, GMM mejora mucho y LDA pasa a ser el mejor clasificador. Esto muestra que los outliers no destruyen la estructura principal, pero sí pueden afectar la comparación entre modelos.